OAVS TGT Computer Science Teacher Examination

Comprehensive MCQ Question Bank with Explanations

Unit: Instructions and Addressing Techniques

Total Questions: **275**

Topics Covered:

Instruction Execution Cycle • Direct Addressing • Indirect Addressing
Relative Addressing • Paging • Indexed Addressing

Definition-based | Conceptual | Application-based | Scenario-based | Statement-based
Assertion-Reason | Match-the-Following | Comparison-based | True/False (MCQ format)
Difficulty Levels: Easy • Moderate • Difficult — Every question includes a short explanation

Table of Contents

1. Instruction Execution Cycle 3
2. Direct Addressing —
3. Indirect Addressing —
4. Relative Addressing —
5. Paging —
6. Indexed Addressing —
7. Comparative & Mixed Addressing Techniques —
8. Answer Key (Quick Reference) —

> Refer to the PDF's footer page numbers for exact section locations.

Section 1: Instruction Execution Cycle

Q1 | **Definition · Easy** | **Instruction Execution Cycle**

What is the basic unit of CPU operation in which one instruction is fetched, decoded and executed?

A. Clock cycle

B. Instruction execution cycle

C. Bus cycle

D. Interrupt cycle

Ans: B

The fetch-decode-execute sequence is the fundamental repeating unit by which a CPU processes every instruction; it is not a clock or bus cycle, which are lower-level timing concepts.

Q2 | **Definition · Easy** | **Instruction Execution Cycle**

The instruction execution cycle is also commonly known as the:

A. Memory cycle

B. Fetch-Decode-Execute cycle

C. I/O cycle

D. DMA cycle

Ans: B

This three-step cycle is universally called the Fetch-Decode-Execute cycle in computer architecture; memory/I/O/DMA cycles refer to specific bus transactions, not the whole instruction cycle.

Q3 | **Conceptual · Easy** | **Instruction Execution Cycle**

Which of the following is the correct sequence of phases in a basic instruction execution cycle?

A. Decode, Fetch, Execute

B. Execute, Fetch, Decode

C. Fetch, Decode, Execute

D. Fetch, Execute, Decode

Ans: C

The CPU must first fetch the instruction from memory, then decode its opcode to understand what to do, and only then execute it — any other order would mean executing before knowing what to execute.

Q4 | **Conceptual · Easy** | **Instruction Execution Cycle**

During the Fetch cycle, the instruction is brought from memory into the:

A. Memory Address Register (MAR)

B. Instruction Register (IR)

C. Accumulator

D. Program Counter (PC)

Ans: B

The Instruction Register (IR) exists specifically to hold the just-fetched instruction so the control unit can decode it; the MAR only holds the address used to fetch it, not the instruction itself.

Q5 | **Definition · Easy** | **Instruction Execution Cycle**

The address of the next instruction to be fetched is held in the:

A. Instruction Register

B. Status Register

C. Program Counter

D. Memory Buffer Register

Ans: C

The Program Counter (PC) always points to the next instruction to be fetched; the IR holds the current instruction, not the next one's address.

Q6 | **Conceptual · Easy** | **Instruction Execution Cycle**

After an instruction is fetched, the Program Counter is normally:

A. Reset to zero

B. Decremented by one

C. Incremented to point to the next instruction

D. Left unchanged

Ans: C

Since instructions are normally executed sequentially, the PC is incremented (by the instruction length) right after fetch so it points to the following instruction, unless a branch overrides this.

Q7 | **Definition · Easy** | **Instruction Execution Cycle**

Which register temporarily holds the instruction currently being decoded/executed?

A. Program Counter

B. Instruction Register

C. Stack Pointer

D. Index Register

Ans: B

The IR is the dedicated register that holds the instruction currently being processed by the control unit during decode and execute; the PC instead tracks the next instruction's address.

Q8 | **Definition · Easy** | **Instruction Execution Cycle**

The phase in which the CPU interprets the opcode of an instruction to determine the operation to be performed is called:

A. Fetch

B. Decode

C. Execute

D. Store

Ans: B

Decoding means interpreting the opcode bits to identify the operation and operands required — this is distinct from fetch (retrieval) and execute (performing the operation).

Q9 | **Conceptual · Easy** | **Instruction Execution Cycle**

In the Execute phase, the CPU:

A. Only reads the next instruction

B. Performs the operation specified by the instruction

C. Clears all registers

D. Halts the processor

Ans: B

By definition, the execute phase is where the operation specified by the decoded instruction is actually carried out, e.g., an addition or a memory write.

Q10 | **Conceptual · Moderate** | **Instruction Execution Cycle**

Which component of the CPU generates control signals to coordinate the fetch, decode and execute phases?

A. ALU

B. Control Unit

C. Cache memory

D. Bus controller

Ans: B

The Control Unit generates the timing and control signals that drive each micro-operation in fetch, decode and execute; the ALU only performs computations, it doesn't sequence the cycle.

Q11 | Assertion-Reason · Moderate | Instruction Execution Cycle

Assertion (A): The Program Counter always holds the address of the instruction currently being executed.
Reason (R): The PC is incremented only after the execute phase is fully completed.
Choose the correct option:

A. Both A and R are true and R correctly explains A

B. Both A and R are true but R does not explain A

C. A is true, R is false

D. A is false, R is true

Ans: D

The PC holds the address of the NEXT instruction once fetch begins (it's already incremented before execute completes), so Assertion A is false; however the second part (R) describing PC behaviour relative to increment timing is essentially true in spirit, making D the best fit among the given assertion-reason pairing.

Q12 | Statement-based · Easy | Instruction Execution Cycle

Statement-based: Consider the following statements about the instruction execution cycle:
(i) Fetch retrieves the instruction from memory.
(ii) Decode interprets the instruction.
(iii) Execute carries out the operation.
Which statements are correct?

A. Only (i)

B. Only (i) and (ii)

C. Only (ii) and (iii)

D. All (i), (ii) and (iii)

Ans: D

All three statements are textbook-accurate descriptions of fetch, decode and execute respectively, so none can be excluded.

Q13 | Definition · Moderate | Instruction Execution Cycle

Which CPU register holds the actual data fetched from or to be written to memory during the cycle?

A. Memory Buffer Register (MBR)/Memory Data Register (MDR)

B. Program Counter

C. Instruction Register

D. Status Flag Register

Ans: A

The MBR/MDR is the register that actually carries data being transferred between the CPU and memory (instructions or operands), distinct from the PC (address of next instruction) or IR (instruction content).

Q14 | Match-the-Following · Moderate | Instruction Execution Cycle

Match the following:
List I: (1) Fetch (2) Decode (3) Execute (4) PC update
List II: (P) Determine operation (Q) Perform operation (R) Get instruction from memory (S) Point to next instruction
Choose the correct match:

A. 1-R, 2-P, 3-Q, 4-S

B. 1-P, 2-R, 3-Q, 4-S

C. 1-R, 2-Q, 3-P, 4-S

D. 1-S, 2-P, 3-Q, 4-R

Ans: A

Fetch retrieves the instruction from memory (R), Decode determines the operation (P), Execute performs the operation (Q), and PC update points to the next instruction (S) — this is the standard role mapping.

Q15 | Conceptual · Moderate | Instruction Execution Cycle

Which of the following best completes the statement? 'The control unit reads the opcode and generates micro-operations during the ___ phase.'

A. Fetch

B. Decode

C. Store

D. Interrupt

Ans: B

By definition, decoding is the step where the control unit interprets the opcode to know what operation and operands are involved.

Q16 | True-False MCQ · Easy | Instruction Execution Cycle

True or False (as MCQ): 'The instruction execution cycle is repeated continuously by the CPU as long as the system is running.' This statement is:

A. True

B. False

C. True only for RISC processors

D. True only during boot-up

Ans: A

The CPU continuously repeats fetch-decode-execute for every instruction as long as it is powered on and running, which is the basic operating principle of a stored-program computer.

Q17 | Conceptual · Moderate | Instruction Execution Cycle

In a typical cycle, fetching the operand from memory (when required) occurs during which broad phase?

A. Fetch instruction phase

B. Decode phase

C. Execute phase (operand fetch sub-step)

D. Interrupt phase

Ans: C

When an instruction needs an operand from memory (e.g., a LOAD), that operand fetch is part of carrying out the instruction's semantics, so it is grouped within the execute phase as an operand-fetch sub-step.

Q18 | Conceptual · Easy | Instruction Execution Cycle

Which of the following is NOT a phase of the basic instruction execution cycle?

A. Fetch

B. Decode

C. Execute

D. Compile

Ans: D

Fetch, decode, and execute are the three universally recognized phases; 'compile' is a software/translation step performed before run-time, not a CPU hardware cycle phase.

Q19 | Definition · Moderate | Instruction Execution Cycle

The Memory Address Register (MAR) is primarily used to:

A. Hold the data to be written to memory

B. Hold the address of the memory location to be accessed

C. Hold the opcode of the instruction

D. Hold the result of ALU operation

Ans: B

The MAR specifically holds the memory address that the CPU wants to read from or write to; it does not store data or instructions itself.

Q20 | Scenario-based · Moderate | Instruction Execution Cycle

Scenario-based: A CPU has just completed decoding an instruction that requires adding two register values. What happens next in the cycle?

A. The CPU fetches the next instruction immediately

B. The CPU executes the addition operation using the ALU

C. The Program Counter is reset

D. The instruction is fetched again

Ans: B

Once an instruction (e.g., ADD) is decoded, the next logical step is to carry out that operation, which is performed by the ALU during execute.

Q21 | Conceptual · Easy | Instruction Execution Cycle

Which unit actually performs the arithmetic/logical operation during the execute phase?

A. Control Unit

B. Arithmetic Logic Unit (ALU)

C. Cache controller

D. Bus arbitration unit

Ans: B The ALU is the functional unit dedicated to arithmetic and logical computations; the control unit only manages signals, it does not compute results.

Q22 | Conceptual · Moderate | Instruction Execution Cycle

If an instruction involves storing a result back to memory, this write-back commonly occurs during the:

A. Fetch phase

B. Decode phase

C. Execute phase

D. Reset phase

Ans: C Writing the computed result back to memory is part of completing the instruction's operation, so it falls within the execute phase (sometimes called execute/write-back).

Q23 | Application-based · Moderate | Instruction Execution Cycle

Application-based: If the Program Counter currently holds address 200, and each instruction is 4 bytes long, what will the PC hold after the next fetch (assuming sequential execution)?

A. 196

B. 200

C. 202

D. 204

Ans: D With PC = 200 and a 4-byte instruction, after fetching that instruction the PC advances by 4 to point to the next instruction at address 204 (200 + 4 = 204).

Q24 | Conceptual · Easy | Instruction Execution Cycle

Which statement about the Instruction Register (IR) is correct?

A. It holds the address of the next instruction

B. It holds the instruction currently being decoded or executed

C. It always holds the result of computation

D. It is updated only at system startup

Ans: B The IR's defined role is to hold whichever instruction the control unit is currently decoding or executing, not the address of the next one.

Q25 | Conceptual · Moderate | Instruction Execution Cycle

The instruction execution cycle is initiated and synchronized by the:

A. System clock

B. Operating system scheduler

C. Compiler

D. Linker

Ans: A

All operations in the cycle (fetch, decode, execute) are timed and triggered by pulses of the system clock, which synchronizes register transfers and control signals.

Q26 | Conceptual · Difficult | Instruction Execution Cycle

Which of these is an additional phase sometimes added to the basic three-phase cycle in advanced CPUs?

A. Fetch operand and write-back

B. Compile and link

C. Format and partition

D. Boot and shutdown

Ans: A

Modern CPUs often add explicit operand-fetch and write-back stages to the basic 3-phase cycle (especially in pipelined designs) to separate memory access from computation.

Q27 | Assertion-Reason · Difficult | Instruction Execution Cycle

Assertion (A): Pipelining overlaps the fetch, decode and execute phases of different instructions.
Reason (R): This overlap is done to increase instruction throughput.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Pipelining works precisely by overlapping the fetch, decode and execute of different instructions simultaneously so that while one instruction executes, the next is being decoded and another fetched — this overlap is the cause of the throughput gain (R correctly explains A).

Q28 | Comparison-based · Moderate | Instruction Execution Cycle

Comparison-based: Compared to a single non-pipelined cycle, a pipelined instruction cycle primarily improves:

A. Instruction size

B. Throughput/Performance

C. Memory capacity

D. Address space

Ans: B

Pipelining does not change instruction size or memory capacity; its main benefit is increased throughput by overlapping multiple instructions' phases.

Q29 | Conceptual · Easy | Instruction Execution Cycle

Which of the following correctly represents the role of the CPU during instruction processing?

A. It only stores data permanently

B. It fetches, decodes and executes instructions, coordinating all operations

C. It only manages peripheral devices

D. It only compiles high-level code

Ans: B

The CPU's central role is exactly this: continuously fetching, decoding, and executing instructions while coordinating registers, the ALU, and memory access through the control unit.

Q30 | Conceptual · Easy | Instruction Execution Cycle

Which of the following best describes the role of the system clock in the instruction execution cycle?

A. It stores the instruction permanently

B. It provides timing pulses that synchronize fetch, decode and execute operations

C. It performs arithmetic operations

D. It holds the opcode of the current instruction

Ans: B

The system clock generates regular timing pulses that drive register transfers and control signals, keeping fetch, decode and execute synchronized.

Q31 | Conceptual · Easy | Instruction Execution Cycle

In a non-pipelined CPU, how many instructions are typically being processed (fetch/decode/execute) at any single instant?

A. Only one

B. Exactly two

C. As many as the cache can hold

D. Unlimited

Ans: A

In a non-pipelined design, the CPU completes one instruction's full fetch-decode-execute sequence before starting the next, so only one instruction is in process at a time.

Q32 Conceptual · Moderate — Instruction Execution Cycle

Which of the following sequences correctly shows what happens when the CPU encounters a HALT instruction during execute?

A. The CPU continues fetching the next instruction normally

B. The CPU stops the fetch-decode-execute cycle until restarted/interrupted

C. The CPU increments PC twice

D. The CPU clears the cache only

Ans: B

A HALT instruction, once executed, stops the normal repeating fetch-decode-execute cycle until an external event (like a reset or interrupt) restarts the CPU.

Q33 Conceptual · Moderate — Instruction Execution Cycle

During the fetch phase, which bus is primarily used to carry the instruction's address from the CPU to memory?

A. Data bus

B. Address bus

C. Control bus

D. I/O bus

Ans: B

The address bus carries the memory address (such as the PC's value) from the CPU to memory so the corresponding instruction can be retrieved.

Q34 Application-based · Moderate — Instruction Execution Cycle

Application-based: If an 8-bit opcode allows for 256 distinct instructions, and decode hardware must distinguish all of them, what is the minimum number of opcode bits required to represent 256 unique instructions?

A. 4 bits

B. 6 bits

C. 8 bits

D. 16 bits

Ans: C

An 8-bit opcode field can directly represent 2^8 = 256 distinct codes, so exactly 8 bits are needed (and sufficient) for 256 instructions.

Q35 | **True-False MCQ · Easy** | **Instruction Execution Cycle**

True/False as MCQ: 'The decode phase can determine the addressing mode used by an instruction.' This statement is:

A. True

B. False

C. True only for indirect addressing

D. True only in 8-bit CPUs

Ans: A

Decoding the opcode also reveals which addressing mode each operand field should be interpreted with, so this is indeed true.

Q36 | **Conceptual · Moderate** | **Instruction Execution Cycle**

Which statement is correct regarding interrupts and the instruction execution cycle?

A. Interrupts are checked only once per program, before execution starts

B. The CPU typically checks for pending interrupts at the end of each instruction cycle

C. Interrupts can never occur during instruction execution

D. Interrupts replace the decode phase entirely

Ans: B

Most CPU designs check for pending interrupts at instruction boundaries (typically after execute), allowing the current instruction to finish cleanly first.

Q37 | **Scenario-based · Moderate** | **Instruction Execution Cycle**

Scenario-based: A CPU fetches an instruction, decodes it as a conditional branch, and the condition evaluates to true during execute. What happens to the PC?

A. It remains unchanged

B. It is loaded with the branch target address instead of simply incrementing

C. It is reset to 0

D. It is incremented twice

Ans: B

When a conditional branch's condition is true, the execute phase loads the PC with the branch target address instead of letting it simply increment.

Q38 | Assertion-Reason · Moderate | Instruction Execution Cycle

Assertion (A): The decode phase must complete before the execute phase begins.
Reason (R): The control unit needs to know which operation and operands are involved before it can carry out the operation.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

The decode phase must finish identifying the operation and operands first, because the execute phase cannot correctly carry out an operation it hasn't yet identified — R correctly explains A.

Q39 | Definition · Easy | Instruction Execution Cycle

Which of the following is the correct definition of an 'instruction cycle time'?

A. The time taken to compile a program

B. The total time taken to fetch, decode and execute one instruction

C. The time taken to boot the operating system

D. The time between two interrupts

Ans: B

Instruction cycle time is, by definition, the total elapsed time required to complete one full fetch-decode-execute sequence for a single instruction.

Q40 | Comparison-based · Moderate | Instruction Execution Cycle

Comparison-based: Compared to a single-cycle (non-pipelined) CPU, a multi-stage pipelined CPU primarily differs in that:

A. It executes only one instruction per program

B. Multiple instructions are in different phases of the cycle simultaneously

C. It cannot use a Program Counter

D. It eliminates the need for decode

Ans: B

Pipelining's key structural difference is that different instructions occupy different phases (fetch/decode/execute) of the cycle at the same time, unlike a single-cycle design where only one phase is active per instruction at a time.

Q41 | Match-the-Following · Moderate | Instruction Execution Cycle

Match the following:
List I: (1) Fetch (2) Decode (3) Execute (4) Interrupt check
List II: (P) Often performed after execute completes (Q) Retrieves instruction bits (R) Identifies opcode and operands (S) Carries out the operation
Choose the correct match:

A. 1-Q, 2-R, 3-S, 4-P

B. 1-R, 2-Q, 3-S, 4-P

C. 1-Q, 2-S, 3-R, 4-P

D. 1-P, 2-Q, 3-R, 4-S

Ans: A

Fetch retrieves the instruction bits (Q), Decode identifies the opcode/operands (R), Execute carries out the operation (S), and the interrupt check commonly happens after execute completes (P) — matching 1-Q, 2-R, 3-S, 4-P.

Q42 | Statement-based · Moderate | Instruction Execution Cycle

Statement-based: Consider the following statements:
(i) The control unit decodes instructions using the opcode field.
(ii) The ALU decodes instructions.
(iii) The execute phase may involve the ALU, memory, or I/O depending on the instruction.
Which are correct?

A. (i) and (iii) only

B. (ii) and (iii) only

C. (i) and (ii) only

D. All three

Ans: A

The control unit (not the ALU) decodes instructions using the opcode field, and the execute phase can indeed involve the ALU, memory, or I/O depending on the instruction type — so (i) and (iii) are correct, (ii) is false.

Q43 | Conceptual · Easy | Instruction Execution Cycle

Which of the following best explains why the fetch phase is sometimes called the 'instruction fetch' (IF) stage in pipelined processor terminology?

A. Because it only fetches data, not instructions

B. Because its specific job is to retrieve the instruction word from memory into the CPU

C. Because it performs arithmetic

D. Because it updates the cache only

Ans: B

The fetch stage's specific responsibility is to retrieve the instruction word from memory and bring it into the CPU (commonly into the IR) for subsequent decoding.

Q44 | **Application-based · Moderate** | **Instruction Execution Cycle**

Application-based: A non-pipelined CPU takes 1 cycle each for fetch, decode and execute. How many clock cycles are needed to completely process 5 sequential instructions (no overlap)?

A. 5

B. 10

C. 15

D. 20

Ans: C

With 1 cycle each for fetch, decode and execute and no overlap, 5 instructions need 5 × 3 = 15 cycles total.

Q45 | **Conceptual · Easy** | **Instruction Execution Cycle**

Which of the following would most directly cause the instruction execution cycle to deviate from simple sequential PC increments?

A. A branch or jump instruction

B. A simple addition instruction

C. A NOP (no operation) instruction

D. Reading the same memory location twice

Ans: A

Branch and jump instructions are the primary cause of non-sequential PC changes, since they explicitly redirect program flow rather than simply incrementing.

Q46 | **Scenario-based · Moderate** | **Instruction Execution Cycle**

Scenario-based: While decoding an instruction, the control unit determines that it is a LOAD instruction requiring a value from memory. What additional action becomes necessary during execute?

A. No additional action is needed

B. A memory read operation must be performed to fetch the operand

C. The PC must be reset to zero

D. The ALU must be disabled

Ans: B

A LOAD instruction's execute phase must include an actual memory read to retrieve the operand value from the address it specifies, beyond just decoding what to do.

Q47 | Conceptual · Difficult | Instruction Execution Cycle

Which of the following is TRUE about the relationship between instruction length and the fetch phase?

A. Longer instructions may require multiple memory fetches to retrieve completely, depending on the architecture

B. Instruction length never affects fetch

C. Fetch only applies to 8-bit instructions

D. Decode always happens before fetch regardless of instruction length

Ans: A

In architectures with variable-length instructions, the fetch unit may need to retrieve additional bytes/words beyond the first fetch to obtain the complete instruction.

Q48 | Conceptual · Difficult | Instruction Execution Cycle

Which of the following best distinguishes 'micro-operations' from the broader fetch-decode-execute phases?

A. Micro-operations are the same as phases, just renamed

B. Micro-operations are the smallest indivisible steps (e.g., register transfers) that make up each broader phase

C. Micro-operations only occur during fetch

D. Micro-operations are performed by the compiler

Ans: B

Micro-operations are the smallest indivisible hardware actions (like a single register transfer) that, when sequenced together, constitute each of the broader fetch, decode, and execute phases.

Q49 | Assertion-Reason · Difficult | Instruction Execution Cycle

Assertion (A): A CPU without an Instruction Register (IR) could not reliably complete the decode phase.
Reason (R): The IR preserves the fetched instruction's bits so the control unit can examine and decode them without being overwritten by the next fetch.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Without an IR to hold the fetched instruction's bits stable, the next fetch could overwrite the data the control unit needs to decode, so the IR's preservation role is essential to reliable decoding — R correctly explains A.

Section 2: Direct Addressing

Q50 | **Definition · Easy** | **Direct Addressing**

In Direct Addressing, the address field of the instruction contains:

A. The operand itself

B. The effective address of the operand

C. The address of the address of the operand

D. An offset to be added to a base register

Ans: B

In Direct Addressing the bits in the address field ARE the effective address of the operand — no further computation or indirection is needed.

Q51 | **Definition · Easy** | **Direct Addressing**

Direct addressing is also referred to as:

A. Immediate addressing

B. Absolute addressing

C. Indirect addressing

D. Indexed addressing

Ans: B

'Absolute addressing' is simply another name for Direct Addressing, because the address used is the actual (absolute) memory location, not relative to anything.

Q52 | **Conceptual · Easy** | **Direct Addressing**

In Direct Addressing, the number of memory references required to access the operand is:

A. Zero

B. One

C. Two

D. Variable, depends on data

Ans: B

Since the address field already IS the effective address, the CPU needs only one memory access — to fetch the operand from that address.

Q53 | Conceptual · Easy | Direct Addressing

Which of the following is a key advantage of Direct Addressing?

A. Very large addressable memory space with a small address field

B. Simplicity and fast execution due to single memory reference

C. Automatic relocation of programs

D. Supports dynamic memory allocation easily

Ans: B Because only a single memory reference is needed (no extra lookup), Direct Addressing is simple to implement and fast to execute.

Q54 | Conceptual · Moderate | Direct Addressing

A major disadvantage of Direct Addressing is:

A. It cannot address memory at all

B. Limited addressable memory because address field size is fixed

C. It always requires two memory accesses

D. It cannot be used in any CPU

Ans: B The address field has a fixed number of bits, so it can only represent a limited range of addresses, capping the directly addressable memory size.

Q55 | Definition · Easy | Direct Addressing

In Direct Addressing, the effective address (EA) is given by:

A. EA = Contents of address field

B. EA = Contents of memory pointed to by address field

C. EA = Base + Displacement

D. EA = Index register + Address field

Ans: A By definition, EA = the value stored in the address field itself — there is no pointer or offset involved.

Q56 | Scenario-based · Moderate | Direct Addressing

Which scenario best illustrates Direct Addressing?

A. The instruction stores a pointer to the operand's address

B. The instruction directly specifies the memory location holding the operand

C. The instruction contains the operand's value itself

D. The instruction adds an index value to a base address

Ans: B Direct Addressing means the instruction's address field names the exact memory location holding the operand, with nothing in between.

Q57 | Statement-based · Moderate | Direct Addressing

Statement-based: Consider the following statements about Direct Addressing:
(i) It requires only one memory access to fetch the operand.
(ii) The address space is limited by the address field width.
(iii) It is the slowest addressing mode due to multiple lookups.
Which are correct?

A. (i) and (ii) only

B. (ii) and (iii) only

C. (i) and (iii) only

D. All three

Ans: A

Statement (iii) is false (direct addressing is actually the fastest, not slowest, due to a single memory reference), so only (i) and (ii), which correctly describe single access and limited range, hold true.

Q58 | Assertion-Reason · Moderate | Direct Addressing

Assertion (A): Direct addressing provides faster operand access compared to indirect addressing.
Reason (R): Direct addressing requires only a single memory reference, while indirect addressing requires two.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Since direct addressing needs only one memory access while indirect addressing needs two (one to get the pointer, one to get the data), direct addressing is inherently faster — R correctly explains A.

Q59 | True-False MCQ · Easy | Direct Addressing

True/False as MCQ: 'In direct addressing, the address field directly gives the effective address of the operand.' This statement is:

A. True

B. False

C. True only for floating-point instructions

D. True only in 8-bit processors

Ans: A

This is the core definition of direct addressing: the address field's contents are taken directly as the effective address of the operand.

Q60 | Conceptual · Moderate | Direct Addressing

Which limitation arises in Direct Addressing when the address field is small (e.g., 8 bits)?

A. Operand values cannot be negative

B. Only a small range of memory locations can be directly addressed

C. Instructions cannot use registers

D. CPU clock speed reduces

Ans: B

A small address field (e.g., 8 bits) can represent only 2^8 = 256 distinct addresses, so only that small range of memory can be reached directly.

Q61 | Application-based · Easy | Direct Addressing

Application-based: If an instruction's address field contains the value 500, and the addressing mode is Direct, where does the CPU fetch the operand from?

A. Register 500

B. Memory location 500

C. Memory location pointed to by contents of address 500

D. Address 500 plus index register value

Ans: B

Since the addressing mode is Direct, the value 500 in the address field is itself the memory location to fetch from — the CPU goes straight to address 500.

Q62 | Conceptual · Moderate | Direct Addressing

Direct addressing is most suitable for:

A. Accessing data structures with dynamic/variable addresses

B. Accessing fixed, known memory locations such as global variables

C. Implementing recursive function calls

D. Relocating programs at run time

Ans: B

Direct addressing works best for data whose location never changes, such as global/static variables known at compile time, since the fixed address is baked into the instruction.

Q63 | Conceptual · Moderate | Direct Addressing

Which of the following is a typical disadvantage that restricts Direct Addressing's use for large programs?

A. It needs a stack pointer

B. Limited address range due to fixed instruction/address field length

C. It cannot be implemented in hardware

D. It requires floating point unit

Ans: B

Because the address field width is fixed by the instruction format, the addressable range cannot grow without changing the instruction set itself — this becomes restrictive for large programs needing access to large memories.

Q64 | Definition · Easy | Direct Addressing

In Direct Addressing mode, the instruction format generally consists of:

A. Opcode + Operand value

B. Opcode + Direct memory address

C. Opcode + Pointer to address

D. Opcode + Base register + Offset

Ans: B

The standard instruction format for direct addressing is: Opcode (what to do) + Direct address (where the operand is) — no extra pointer field is used.

Q65 | Comparison-based · Moderate | Direct Addressing

Comparison-based: Compared to Indirect Addressing, Direct Addressing offers:

A. Larger address range but slower access

B. Smaller address range but faster access (single memory reference)

C. Same address range and same speed

D. No memory reference at all

Ans: B

Direct addressing needs only one memory reference (fast) but the address field's limited width restricts the addressable range, while indirect addressing reaches further at the cost of an extra memory access.

Q66 | Application-based · Moderate | Direct Addressing

Which of these is an application of Direct Addressing in real programs?

A. Accessing array elements via a loop index

B. Accessing a fixed global variable at a known memory location

C. Calling a subroutine via a function pointer

D. Implementing a linked list traversal

Ans: B

Accessing a known, unchanging global variable is a textbook use-case for direct addressing, since its address is fixed and can be embedded directly in instructions.

Q67 | Match-the-Following · Moderate | Direct Addressing

Match the following:
List I: (1) Direct Addressing (2) Single memory reference (3) Fixed address field
List II: (P) Limits address range (Q) Operand fetched in one step (R) Address given directly in instruction
Choose correct match:

A. 1-R, 2-Q, 3-P

B. 1-Q, 2-R, 3-P

C. 1-P, 2-Q, 3-R

D. 1-R, 2-P, 3-Q

Ans: A

Fetch (1) brings the instruction from memory (R), Single memory reference (2) means the operand is obtained in one step (Q), and Fixed address field (3) limits the address range (P) — matching 1-R, 2-Q, 3-P.

Q68 | Conceptual · Moderate | Direct Addressing

Which factor primarily determines the maximum memory that can be directly addressed using Direct Addressing?

A. Clock speed of CPU

B. Width (number of bits) of the address field in the instruction

C. Size of the data bus only

D. Number of general-purpose registers

Ans: B

The number of bits allocated to the address field directly limits how many distinct memory locations can be represented and thus directly addressed (2^n locations for an n-bit field).

Q69 | Conceptual · Easy | Direct Addressing

Direct Addressing is considered a 'one-level' addressing scheme because:

A. It uses two levels of memory access

B. The address field points directly to the operand without further indirection

C. It always uses a register as intermediate

D. It requires paging hardware

Ans: B

Because the address field points straight to the operand with no further address lookup, direct addressing is called a 'one-level' scheme — indirect addressing, by contrast, is two-level.

Q70 | Conceptual · Difficult | Direct Addressing

Which is TRUE regarding flexibility of Direct Addressing for relocatable programs?

A. It easily supports program relocation

B. It is relatively inflexible for relocation since addresses are fixed in the instruction

C. It supports relocation better than indexed addressing

D. It is unrelated to program relocation

Ans: B

Since the address is hard-coded as part of the instruction, relocating a program to a different memory location would require changing every embedded address, making direct addressing relatively inflexible for relocation.

Q71 | Scenario-based · Easy | Direct Addressing

Scenario-based: A compiler generates code that directly embeds the memory address 1024 in an instruction's address field to access variable X. This is an example of:

A. Indirect Addressing

B. Direct Addressing

C. Relative Addressing

D. Indexed Addressing

Ans: B

Embedding the literal address 1024 in the instruction's address field — with no pointer or offset — is the defining behaviour of Direct Addressing.

Q72 | Conceptual · Moderate | Direct Addressing

In terms of execution speed, Direct Addressing is generally:

A. Slower than indirect addressing

B. Faster than indirect addressing because of fewer memory accesses

C. Equal to paging in speed

D. Dependent only on CPU cache size

Ans: B

Direct addressing needs only one memory access to get the operand, whereas indirect addressing needs two (fetch pointer, then fetch operand), making direct addressing faster.

Q73 | Conceptual · Moderate | Direct Addressing

Which of the following is NOT an advantage of Direct Addressing?

A. Simplicity of implementation

B. Fast operand access

C. Large addressable memory space using a short address field

D. Easy to decode by hardware

Ans: C

A large addressable range using a SHORT address field is not possible with direct addressing — that is actually a strength of indirect addressing, which is why it is NOT an advantage of direct addressing.

Q74 | Assertion-Reason · Moderate | Direct Addressing

Assertion (A): Direct Addressing cannot address the entire memory of a large modern computer using a short instruction address field.
Reason (R): The address field length restricts the directly addressable range.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

A short, fixed address field can only span a limited memory range (A is true), and this limitation arises specifically because the field's bit-width caps the maximum representable address (R correctly explains A).

Q75 | Definition · Easy | Direct Addressing

Direct Addressing is best characterized as providing:

A. Two memory accesses and large range

B. One memory access and limited range

C. Zero memory accesses and unlimited range

D. Variable memory accesses depending on page size

Ans: B

Direct addressing's defining trade-off is: exactly one memory access, but the address range is capped by the field's bit-width.

Q76 | Application-based · Moderate | Direct Addressing

Which programming construct is typically implemented efficiently using Direct Addressing?

A. Access to a global counter variable at a fixed location

B. Access to dynamically allocated heap memory

C. Access to recursive stack frames

D. Access to paged virtual memory

Ans: A

A fixed-location global counter is accessed at the same address every time, which is exactly what direct addressing is suited for.

Q77 | Conceptual · Difficult | Direct Addressing

In Direct Addressing, if the operand needs to be changed at run-time to point elsewhere, this is:

A. Easily done since address is in a register

B. Difficult since the address is hard-coded in the instruction

C. Automatically handled by paging hardware

D. Not applicable since there is no address

Ans: B

Since the address is embedded literally in the instruction's address field, changing what it points to at run-time would require modifying the instruction itself, which is impractical — unlike indirect addressing where only the pointer's contents need to change.

Q78 | Conceptual · Moderate | Direct Addressing

Which of the following best explains why Direct Addressing is sometimes avoided in large modern programs with extensive memory needs?

A. It cannot use the ALU

B. Its address field cannot be made long enough without enlarging every instruction, which is impractical

C. It cannot work with the Program Counter

D. It only works with floating-point numbers

Ans: B

Direct addressing's address field has a fixed bit-width set by the instruction format; widening it to cover more memory would require enlarging every instruction, which is generally impractical at scale.

Q79 | Statement-based · Moderate | Direct Addressing

Statement-based: Consider the following about Direct Addressing:
(i) Programs using only direct addressing can still be relocated if the loader rewrites every embedded address.
(ii) Direct addressing has zero overhead beyond a single memory access.
(iii) Direct addressing supports unlimited address ranges regardless of field size.
Which statements are correct?

A. (i) and (ii) only

B. (ii) and (iii) only

C. (i) and (iii) only

D. All three

Ans: A

Direct addressing only needs a single memory access (ii is true), and a loader CAN make relocation work by rewriting every embedded address (i is true), but the address range is NOT unlimited — it is capped by the field's width, so (iii) is false.

Q80 | Application-based · Moderate | Direct Addressing

Application-based: An instruction format reserves 10 bits for the direct address field. What is the maximum directly addressable memory size?

A. 100 locations

B. 512 locations

C. 1024 locations

D. 2048 locations

Ans: C

A 10-bit field can represent 2^10 = 1024 distinct addresses, giving a maximum of 1024 directly addressable locations.

Q81 | Scenario-based · Moderate | Direct Addressing

Which of these scenarios is LEAST suited to Direct Addressing?

A. Accessing a compile-time constant stored at a fixed location

B. Accessing the i-th element of a dynamically sized array inside a loop

C. Accessing a global counter variable

D. Accessing a fixed configuration register

Ans: B

Accessing an array element whose index changes each loop iteration needs the address to change dynamically, which direct addressing (a fixed, hard-coded address) cannot do efficiently — indexed addressing is the natural fit instead.

Q82 | Assertion-Reason · Moderate | Direct Addressing

Assertion (A): Direct addressing instructions are typically decoded faster than indirect addressing instructions.
Reason (R): Direct addressing requires no additional pointer-dereferencing step before the effective address is known.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Because direct addressing's effective address is simply the address field's value (no pointer to dereference), decoding and computing the effective address is quicker than indirect addressing, which needs an extra dereferencing step — R correctly explains A.

Q83 | Conceptual · Easy | Direct Addressing

Which of the following is an accurate one-line summary of Direct Addressing's central limitation?

A. It cannot be implemented in hardware

B. Its reach is capped by the width of the instruction's address field

C. It always needs two memory accesses

D. It cannot store negative numbers

Ans: B

Direct addressing's central trade-off is that the directly reachable memory range is bounded by however many bits the instruction's address field has.

Q84 | True-False MCQ · Moderate | Direct Addressing

True/False as MCQ: 'Direct addressing can address more memory than indirect addressing for the same instruction word size.' This statement is:

A. True

B. False — indirect addressing can reach a larger range via the pointed-to word

C. True only for 64-bit systems

D. True only when paging is disabled

Ans: B

Indirect addressing can reach a much larger effective address space because the pointed-to memory word can store a full-width address, unlike direct addressing's address field, which is limited by the instruction's fixed width — so the statement is false.

Q85 | Match-the-Following · Moderate | Direct Addressing

Match the following:
List I: (1) Direct Addressing (2) Address field (3) EA computation
List II: (P) No computation needed, EA = field value (Q) Holds the literal memory address (R) Direct Addressing's defining mode
Choose correct match:

A. 1-R, 2-Q, 3-P

B. 1-Q, 2-R, 3-P

C. 1-P, 2-Q, 3-R

D. 1-R, 2-P, 3-Q

Ans: A

The address field (2) holds the literal memory address (Q), and this is exactly Direct Addressing's defining mode (3=R), where no extra computation beyond reading the field is needed (1=P) — matching 1-R, 2-Q, 3-P.

Q86 | Comparison-based · Moderate | Direct Addressing

Comparison-based: For accessing the SAME fixed memory cell repeatedly inside a tight loop, which addressing mode introduces the least overhead?

A. Indirect Addressing

B. Direct Addressing

C. Indexed Addressing with a changing index

D. Multi-level Indirect Addressing

Ans: B

Since direct addressing needs only one memory access with no pointer dereference, it introduces the least overhead for repeatedly accessing the same fixed memory cell.

Q87 | Scenario-based · Easy | Direct Addressing

Scenario-based: A compiler assigns the address 4096 to a global variable 'total' and embeds this address directly into every instruction that reads or writes 'total'. This illustrates:

A. Indirect Addressing

B. Relative Addressing

C. Direct Addressing

D. Indexed Addressing

Ans: C

Embedding a fixed, known memory address (4096) directly into every instruction referencing that variable is the textbook behaviour of Direct Addressing.

Section 3: Indirect Addressing

Q88 | **Definition · Easy** | **Indirect Addressing**

In Indirect Addressing, the address field of the instruction contains:

A. The operand itself

B. The effective address of the operand directly

C. The address of a memory location that holds the effective address

D. An index value to be added to base register

Ans: C

In Indirect Addressing, the address field does not give the operand's address directly — instead it gives the address of a memory location that itself holds the effective address (a pointer).

Q89 | **Conceptual · Easy** | **Indirect Addressing**

How many memory references are typically required to fetch an operand using Indirect Addressing?

A. Zero

B. One

C. Two

D. Three or more, always

Ans: C

The CPU must first read the pointer from the location named in the address field, and then use that pointer's value to access the actual operand — that is two memory references.

Q90 | **Definition · Easy** | **Indirect Addressing**

In Indirect Addressing, the effective address is obtained as:

A. EA = Address field

B. EA = Contents of the memory location specified by the address field

C. EA = Base + Index

D. EA = PC + Displacement

Ans: B

By definition, EA = the value found by reading the memory location specified by the instruction's address field (i.e., dereferencing the pointer).

Q91 | Conceptual · Moderate | Indirect Addressing

A key advantage of Indirect Addressing is:

A. It always requires only one memory access

B. It allows a very large effective address space since the address field points to a full-length address elsewhere

C. It eliminates the need for a memory at all

D. It is faster than direct addressing

Ans: B

Because the address field only needs to point to a memory word that can hold a full address, the effective address space reachable is determined by that word's width, not the (possibly short) instruction address field — giving a much larger range.

Q92 | Conceptual · Moderate | Indirect Addressing

A major disadvantage of Indirect Addressing is:

A. It cannot reference memory beyond the instruction

B. It requires multiple memory accesses, making it slower than direct addressing

C. It cannot be used with pointers

D. It is unsupported in modern CPUs

Ans: B

The extra step of first reading the pointer before reaching the operand means two memory accesses are needed instead of one, making indirect addressing slower than direct addressing.

Q93 | Conceptual · Easy | Indirect Addressing

Indirect Addressing is closely associated with the concept of:

A. Immediate operands

B. Pointers (address of an address)

C. Relative offsets

D. Page tables only

Ans: B

Indirect addressing is essentially the hardware equivalent of a pointer: 'the address field holds the address of the address' of the actual data.

Q94 | Scenario-based · Easy | Indirect Addressing

Scenario-based: An instruction's address field contains 300. Memory location 300 contains the value 700, which is the actual address of the operand. This describes:

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Immediate Addressing

Ans: B

Here, the address field (300) names a location whose stored content (700) is then used as the address of the actual operand — this two-step lookup is exactly indirect addressing.

Q95 | Statement-based · Moderate | Indirect Addressing

Statement-based: Consider the following about Indirect Addressing:
(i) It uses two memory references to access the operand.
(ii) The address field holds a pointer to the operand's address.
(iii) It is always faster than direct addressing.
Which statements are correct?

A. (i) and (ii) only

B. (ii) and (iii) only

C. (i) and (iii) only

D. All three

Ans: A

Statement (iii) is false because indirect addressing is slower, not faster, than direct addressing due to the extra memory reference; only (i) and (ii) are accurate.

Q96 | Assertion-Reason · Difficult | Indirect Addressing

Assertion (A): Indirect addressing can support a much larger effective address space than direct addressing for the same instruction address field size.
Reason (R): The address field in indirect addressing points to a memory word that can hold a full address, rather than the operand's address directly limited by field width.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Because the pointer location can store a full-width address (not limited by the instruction's address field size), indirect addressing can reach a far larger effective address space — R correctly explains A.

Q97 | True-False MCQ · Easy | Indirect Addressing

True/False as MCQ: 'Indirect addressing always results in faster execution than direct addressing.' This statement is:

A. True

B. False

C. True only for branch instructions

D. True only in RISC architecture

Ans: B

Indirect addressing requires an extra memory access to fetch the pointer before getting the operand, so it is generally slower, not faster, than direct addressing — the statement is false.

Q98 | Conceptual · Difficult | Indirect Addressing

Multi-level indirect addressing refers to:

A. A single memory reference to get the operand

B. A chain of pointers where one address leads to another address, and so on, before reaching the operand

C. Direct access without any pointer

D. Using index registers exclusively

Ans: B

Multi-level indirect addressing means the pointer itself can point to another pointer, and so on, through a chain, before the final operand address is reached.

Q99 | Application-based · Moderate | Indirect Addressing

Application-based: If memory location 50 (given in the address field) contains 900, and location 900 contains the actual data 42, using simple (single-level) Indirect Addressing, what is the effective address?

A. 50

B. 900

C. 42

D. Cannot be determined

Ans: B

With single-level indirect addressing, the address field (50) points to a location whose stored value (900) IS the effective address — so EA = 900, and the operand 42 is fetched from there.

Q100 | Application-based · Moderate | Indirect Addressing

Which of the following is an application where Indirect Addressing is commonly used?

A. Implementing pointers and dynamic data structures (e.g., linked lists)

B. Accessing fixed global constants

C. Encoding immediate numeric literals

D. Implementing simple loop counters only

Ans: A

Indirect addressing's 'address of an address' mechanism mirrors how pointers work in programming, making it natural for implementing pointer-based structures like linked lists.

Q101 | Comparison-based · Moderate | Indirect Addressing

Compared to Direct Addressing, Indirect Addressing trades off:

A. Speed for larger addressable range

B. Range for higher speed

C. Nothing; they are identical in performance

D. Memory for CPU registers

Ans: A

Indirect addressing trades a slower lookup (one extra memory access) for the benefit of being able to reach a much larger address range than the fixed instruction field would otherwise allow.

Q102 | Match-the-Following · Moderate | Indirect Addressing

Match the following:
List I: (1) Direct (2) Indirect (3) Pointer chain
List II: (P) Multi-level indirect addressing (Q) Single memory reference (R) Address field gives address of address
Choose correct match:

A. 1-Q, 2-R, 3-P

B. 1-R, 2-Q, 3-P

C. 1-P, 2-R, 3-Q

D. 1-Q, 2-P, 3-R

Ans: A

Direct (1) uses one reference (Q), Indirect (2) uses the address field to give the address of the address (R), and a Pointer chain (3) describes multi-level indirection (P) — matching 1-Q, 2-R, 3-P.

Q103 | Conceptual · Moderate | Indirect Addressing

In Indirect Addressing, the register or memory location that holds the actual address of the operand is sometimes called:

A. Index register

B. Pointer (or indirect address holder)

C. Stack pointer only

D. Program counter

Ans: B

The memory location (or register) that stores the actual address of the operand, used as the intermediate step in indirect addressing, is generally called a pointer.

Q104 | Conceptual · Difficult | Indirect Addressing

Which problem can occur if multi-level indirect addressing is used excessively?

A. Increased speed of execution

B. Increased number of memory accesses, reducing performance

C. Decreased addressable memory space

D. Loss of data type information

Ans: B

Chaining multiple levels of indirection means the CPU must perform an extra memory access at each level before finally reaching the operand, which compounds performance overhead.

Q105 | Conceptual · Moderate | Indirect Addressing

Indirect addressing is useful when:

A. The operand's address is fixed and known at compile time

B. The operand's address may change at run time or needs to be computed dynamically

C. No memory access is required

D. The instruction set has no address field

Ans: B

Indirect addressing is valuable exactly when the operand's true location is not fixed and may need to be computed or changed at run time, since only the pointer's stored value needs to change.

Q106 | Conceptual · Easy | Indirect Addressing

Which addressing mode requires the CPU to access memory twice (in its simplest single-level form) to obtain the operand?

A. Direct Addressing

B. Indirect Addressing

C. Immediate Addressing

D. None of the above

Ans: B

In its simplest (single-level) form, indirect addressing requires the CPU to read memory once to get the pointer and a second time to get the actual operand — two accesses total.

Q107 | Assertion-Reason · Moderate | Indirect Addressing

Assertion (A): Indirect addressing is slower than direct addressing for operand access.
Reason (R): It requires an extra memory access to first retrieve the effective address before retrieving the operand.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Because indirect addressing requires an extra memory reference to first retrieve the pointer, it is inherently slower than direct addressing, which fetches the operand directly — R correctly explains A.

Q108 | Conceptual · Easy | Indirect Addressing

In high-level languages, indirect addressing corresponds conceptually to the use of:

A. Constants

B. Pointers/references

C. Loop counters

D. Comments

Ans: B

A pointer or reference variable in a high-level language is conceptually exactly what indirect addressing implements at the hardware level: a stored value that is itself an address.

Q109 | Scenario-based · Moderate | Indirect Addressing

Scenario-based: A function call uses a value stored in a register that itself contains the address of the actual subroutine to call. This is best described as:

A. Direct Addressing

B. Indirect Addressing

C. Paging

D. Relative Addressing

Ans: B

Using a register that contains the address of the subroutine (rather than the subroutine's address being given directly) is a classic example of indirect addressing applied to control transfer.

Q110 | Conceptual · Moderate | Indirect Addressing

Which of these is generally considered a disadvantage unique to Indirect Addressing compared to Direct Addressing?

A. Smaller address range

B. Extra memory access overhead

C. Inability to use registers

D. Inability to support pointers

Ans: B

The defining cost of indirect addressing relative to direct addressing is the extra memory access needed to dereference the pointer before reaching the operand.

Q111 | Definition · Easy | Indirect Addressing

The 'address of address' concept is the defining characteristic of:

A. Relative Addressing

B. Indexed Addressing

C. Indirect Addressing

D. Paging

Ans: C

By definition, indirect addressing is the technique where the instruction's address field gives the address OF the address of the operand — this 'address of address' idea is its hallmark.

Q112 | Comparison-based · Easy | Indirect Addressing

Which statement correctly differentiates Direct and Indirect Addressing?

A. Direct gives the operand's address directly; Indirect gives the address of a location holding the operand's address

B. Direct always uses two memory accesses; Indirect uses one

C. Both require the same number of memory accesses

D. Indirect addressing cannot use memory at all

Ans: A

Direct addressing gives the operand's address straight away, while indirect addressing gives the address of a pointer that must be dereferenced to find the operand's address — this is the key structural difference.

Q113 | Conceptual · Moderate | Indirect Addressing

In Indirect Addressing, if the pointer location's content is updated at run-time, the actual operand referenced:

A. Remains unchanged regardless of pointer content

B. Changes accordingly, since the effective address depends on the pointer's content

C. Cannot be changed at all

D. Always becomes zero

Ans: B

Since the effective address is computed by reading the pointer location's current contents, any run-time change to that content automatically changes which operand is accessed.

Q114 | Application-based · Moderate | Indirect Addressing

Which of the following best supports implementation of dynamic memory allocation routines?

A. Direct Addressing

B. Indirect Addressing

C. Immediate Addressing

D. None of the above

Ans: B

Indirect addressing (via pointers) is the natural mechanism for dynamic memory allocation, since allocated blocks' addresses are stored in pointers rather than fixed in code.

Q115 | True-False MCQ · Moderate | Indirect Addressing

True/False as MCQ: 'In indirect addressing, the instruction's address field never directly contains the operand or its final address.' This statement is:

A. True

B. False

C. True only for negative operands

D. True only in 32-bit systems

Ans: A

By definition, the address field in indirect addressing never IS the final address or the operand — it always names a location holding that address, so the statement is true.

Q116 | Conceptual · Difficult | Indirect Addressing

Which of the following is an accurate statement about double (two-level) indirect addressing?

A. It requires the same number of memory accesses as direct addressing

B. It requires three memory accesses: one for each level of pointer plus the final operand

C. It requires zero memory accesses

D. It is faster than single-level indirect addressing

Ans: B

Double indirect addressing involves reading the first pointer, then the second pointer it leads to, and finally the operand itself — three separate memory accesses in total.

Q117 | Application-based · Moderate | Indirect Addressing

Application-based: Address field value = 10. Memory[10] = 50. Memory[50] = 99. Using single-level indirect addressing, what is the operand fetched?

A. 10

B. 50

C. 99

D. Cannot be determined

Ans: C

Memory[10] = 50 gives the address of the next lookup; Memory[50] = 99 is the value at that address, so the final operand fetched is 99.

Q118 | Assertion-Reason · Moderate | Indirect Addressing

Assertion (A): Indirect addressing is preferred when a subroutine must work with different data each time it is called via a parameter passed by reference.
Reason (R): The pointer (address) can simply be updated to refer to new data without changing the subroutine's instructions.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Indirect addressing lets a subroutine work with different data on each call simply by changing what the pointer refers to, without altering any instruction — R correctly explains why this technique suits call-by-reference.

Q119 | Statement-based · Moderate | Indirect Addressing

Statement-based: Consider the following about indirect addressing:
(i) The number of memory accesses grows with the number of indirection levels.
(ii) It always uses an index register.
(iii) It is conceptually equivalent to dereferencing a pointer.
Which statements are correct?

A. (i) and (iii) only

B. (ii) and (iii) only

C. (i) and (ii) only

D. All three

Ans: A

Indirection levels do add proportionally more memory accesses (i is true), and the technique is conceptually a hardware analogue of pointer dereferencing (iii is true), but indirect addressing does not require an index register, so (ii) is false.

Q120 | Application-based · Moderate | Indirect Addressing

Which of the following is the most accurate reason indirect addressing is used in call-by-reference parameter passing?

A. Because it is the fastest addressing mode available

B. Because the called function can access and modify the caller's actual variable via its address rather than a copy

C. Because it eliminates the need for a stack

D. Because it doesn't require any addressing mode at all

Ans: B

Passing a reference (address) lets the called function operate directly on the caller's actual variable rather than a separate copy, which is precisely what indirect addressing enables at the hardware level.

Q121 | True-False MCQ · Easy | Indirect Addressing

True/False as MCQ: 'In indirect addressing, changing the operand referenced requires modifying the instruction's opcode.' This statement is:

A. True

B. False — only the pointer's stored value needs to change, not the instruction

C. True only for direct addressing

D. True only for relative addressing

Ans: B

In indirect addressing, only the contents of the pointer location need to be updated to change which operand is accessed; the instruction's opcode and address field remain unchanged — so the statement is false.

Q122 | Comparison-based · Easy | Indirect Addressing

Comparison-based: Which requires fewer memory accesses to fetch an operand: direct addressing or single-level indirect addressing?

A. Direct addressing (one access) requires fewer than indirect addressing (two accesses)

B. Indirect addressing requires fewer accesses

C. Both require the same number of accesses

D. Neither requires any memory access

Ans: A

Direct addressing needs just one memory access for the operand, while (single-level) indirect addressing needs two — first the pointer, then the operand — so direct addressing requires fewer accesses.

Q123 | Match-the-Following · Moderate | Indirect Addressing

Match the following:
List I: (1) Indirect Addressing (2) Pointer (3) Two memory accesses
List II: (P) Holds the address of the operand (Q) Characteristic cost of indirect addressing (R) Address field gives address of address
Choose the correct match:

A. 1-R, 2-P, 3-Q

B. 1-P, 2-R, 3-Q

C. 1-Q, 2-P, 3-R

D. 1-R, 2-Q, 3-P

Ans: A

Indirect Addressing (1) is defined by the address field giving the address of the address (R), the pointer (2) holds the operand's actual address (P), and the resulting cost is two memory accesses (3=Q) — matching 1-R, 2-P, 3-Q.

Q124 | **Scenario-based · Easy** | **Indirect Addressing**

Scenario-based: A linked list's 'next' field in each node stores the memory address of the following node. Traversing the list by following these addresses is best described as repeated use of:

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Paging

Ans: B

Following a 'next' pointer stored in each node to reach the following node is exactly the address-of-an-address mechanism that defines indirect addressing.

Q125 | **Conceptual · Moderate** | **Indirect Addressing**

Which of the following best explains why excessive levels of indirect addressing are avoided in performance-critical code?

A. Each additional level adds another memory access, increasing latency

B. It uses more opcode bits than other modes

C. It cannot be implemented in any CPU

D. It disables the Program Counter

Ans: A

Each additional level of indirection adds one more memory access before the operand is finally reached, which directly increases latency in time-critical code.

Section 4: Relative Addressing

Q126 | Definition · Easy | Relative Addressing

In Relative Addressing, the effective address is computed as:

A. Base register + Index register

B. Program Counter (PC) + Displacement (offset) given in the instruction

C. Address field value directly

D. Contents of memory pointed to by address field

Ans: B

Relative Addressing computes the effective address by adding a signed displacement (given in the instruction) to the current Program Counter value — EA = PC + displacement.

Q127 | Conceptual · Moderate | Relative Addressing

Relative Addressing is primarily used to support:

A. Fixed global variable access

B. Position-independent code and short branch/jump instructions

C. Multi-level pointer chains

D. Paged virtual memory translation

Ans: B

Because the target address is expressed relative to the current instruction's location rather than as a fixed absolute address, the same code works correctly wherever it is loaded — ideal for relocatable code and short branches.

Q128 | Conceptual · Moderate | Relative Addressing

In Relative Addressing, the displacement (offset) value in the instruction is typically:

A. Always positive only

B. A signed value that can be positive or negative

C. Always equal to the operand value

D. Always zero

Ans: B

The displacement must be signed (positive or negative) so that branches can jump both forward and backward in memory relative to the current PC.

Q129 | Conceptual · Moderate | Relative Addressing

A major advantage of Relative Addressing is:

A. It requires two memory accesses always

B. It allows a smaller number of bits to represent a wide effective address range relative to the current instruction

C. It cannot be used for branching

D. It requires absolute addresses to be hardcoded

Ans: B

A small displacement field can express a wide range of nearby target addresses purely relative to the PC, without needing a full-width absolute address field.

Q130 | Application-based · Easy | Relative Addressing

Relative Addressing is especially useful for implementing:

A. Global constant tables only

B. Conditional and unconditional branch/jump instructions

C. Static memory allocation only

D. I/O port mapping only

Ans: B

Relative addressing is the natural choice for branch/jump instructions, since most branch targets are 'nearby' in the code and can be expressed as small offsets from the current instruction.

Q131 | Application-based · Moderate | Relative Addressing

Scenario-based: An instruction at memory address 1000 contains a displacement of +20 and uses relative addressing relative to the PC (which after fetch points to 1004). What is the effective address?

A. 1000

B. 1004

C. 1020

D. 1024

Ans: D

With PC = 1004 (after fetch) and displacement +20, EA = 1004 + 20 = 1024.

Q132 | Definition · Easy | Relative Addressing

Which register is essential to compute the effective address in Relative Addressing?

A. Stack Pointer

B. Program Counter

C. Index Register

D. Memory Buffer Register

Ans: B

The PC's current value is essential to relative addressing because the effective address is always computed as an offset from it; without the PC there is no reference point.

Q133 | Statement-based · Moderate | Relative Addressing

Statement-based: Consider the following statements about Relative Addressing:
(i) The effective address depends on the current value of the PC.
(ii) It supports position-independent code.
(iii) The displacement field is generally smaller than a full absolute address.
Which are correct?

A. (i) and (ii) only

B. (i), (ii) and (iii)

C. (ii) and (iii) only

D. (i) and (iii) only

Ans: B

All three statements correctly describe relative addressing: it depends on PC (i), enables position-independence (ii), and uses a smaller field than an absolute address would need (iii).

Q134 | Assertion-Reason · Moderate | Relative Addressing

Assertion (A): Relative addressing allows programs to be relocated in memory without modifying address references.
Reason (R): The effective address is computed relative to the current PC value rather than as a fixed absolute address.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Because the offset is relative to the current PC rather than a fixed memory location, the same instruction works no matter where the program is loaded in memory — R correctly explains why relocation works.

Q135 | True-False MCQ · Easy | Relative Addressing

True/False as MCQ: 'Relative addressing computes the effective address using a fixed memory location unrelated to the program counter.' This statement is:

A. True

B. False

C. True for indirect addressing only

D. True for paging only

Ans: B

Relative addressing computes EA using PC + displacement, which is NOT a fixed memory location — so the statement describing it as fixed and unrelated to PC is false.

Q136 | Conceptual · Moderate | Relative Addressing

Which of the following is a disadvantage of Relative Addressing?

A. It cannot support branching instructions

B. The limited range of the displacement field restricts how far (in memory) the effective address can be from the PC

C. It always needs two memory references

D. It cannot be implemented in hardware

Ans: B

The displacement field has a limited number of bits, so it can only express targets within a certain distance from the current PC — distant targets cannot be reached without extra instructions.

Q137 | Application-based · Easy | Relative Addressing

Relative Addressing is most commonly applied in:

A. Storing large arrays of fixed data

B. Short-distance jump/branch instructions in a program's control flow

C. Operating system page tables

D. Direct I/O port addressing

Ans: B

Branch and jump instructions overwhelmingly target nearby code, making relative addressing's compact, PC-relative encoding the standard choice for them.

Q138 | Comparison-based · Moderate | Relative Addressing

Comparison-based: Compared to Direct Addressing, Relative Addressing's effective address depends on:

A. A fixed absolute address only

B. Both the instruction's displacement and the current PC value

C. Only the contents of a separate memory location

D. An index register and a base register

Ans: B

Unlike direct addressing's fixed absolute address, relative addressing's EA depends on both the instruction's encoded displacement AND wherever the PC currently happens to be.

Q139 | Conceptual · Moderate | Relative Addressing

Which property makes relative addressing valuable for writing position-independent/relocatable code?

A. Addresses computed relative to current execution point rather than absolute memory locations

B. Use of two-level indirection

C. Direct hardcoding of absolute addresses

D. Use of page tables

Ans: A

Because addresses are computed relative to the current execution point (PC) rather than as fixed absolute locations, code can be loaded anywhere in memory and still execute correctly — this is the basis of position-independent code.

Q140 | Conceptual · Moderate | Relative Addressing

In Relative Addressing for branch instructions, a negative displacement typically means:

A. The branch target is at a higher memory address than the current instruction

B. The branch target is at a lower (earlier) memory address than the current instruction

C. The instruction is invalid

D. The branch is unconditional

Ans: B

A negative displacement, when added to the PC, results in an address smaller than the current PC, i.e., a target earlier (lower) in memory.

Q141 | Match-the-Following · Moderate | Relative Addressing

Match the following:
List I: (1) Relative Addressing (2) Displacement (3) Position-independent code
List II: (P) Benefit of relative addressing (Q) Offset added to PC (R) EA = PC + Offset
Choose correct match:

A. 1-R, 2-Q, 3-P

B. 1-Q, 2-R, 3-P

C. 1-P, 2-Q, 3-R

D. 1-R, 2-P, 3-Q

Ans: A

Relative Addressing (1) computes EA = PC + Offset (R), Displacement (2) is the offset added to PC (Q), and Position-independent code (3) is the benefit gained (P) — matching 1-R, 2-Q, 3-P.

Q142 | Application-based · Moderate | Relative Addressing

Application-based: A CPU's PC after fetch is 2048. A branch instruction uses relative addressing with displacement -8. What is the target effective address?

A. 2040

B. 2048

C. 2056

D. 2000

Ans: A

With PC = 2048 and displacement -8, EA = 2048 + (-8) = 2040.

Q143 Application-based · Moderate — Relative Addressing

Which of the following addressing modes is generally preferred for short-range conditional jumps in assembly language due to compact instruction encoding?

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Paging

Ans: C

Relative addressing's compact, PC-relative encoding is the standard mechanism compilers/assemblers use for short conditional and unconditional jumps.

Q144 Conceptual · Difficult — Relative Addressing

In Relative Addressing, if the displacement field has 'n' bits (signed), the range of addresses reachable relative to PC is approximately:

A. 0 to 2^n

B. -2^(n-1) to +2^(n-1)-1

C. Unlimited

D. Always exactly 256 locations

Ans: B

A signed n-bit displacement field can represent values from -2^(n-1) to +2^(n-1)-1, giving the reachable range of addresses relative to PC.

Q145 Comparison-based · Moderate — Relative Addressing

Which is TRUE about Relative Addressing compared to Direct Addressing regarding relocation?

A. Direct addressing relocates more easily than relative addressing

B. Relative addressing supports easier relocation since addresses are PC-relative, not absolute

C. Both are equally easy to relocate

D. Neither supports relocation

Ans: B

Because relative addressing expresses target addresses as offsets from the current PC rather than as fixed absolute locations, the same code can be relocated to a new memory region and the offsets remain valid.

Q146 | Assertion-Reason · Difficult | Relative Addressing

Assertion (A): Relative addressing cannot be used for accessing data, only for branching.
Reason (R): The PC is only updated during branch instructions.
Choose the correct option regarding the assertion (not the reason's truth):

A. A is correct: relative addressing is exclusively for branching

B. A is incorrect: relative addressing can also be used to access data relative to the PC

C. A is correct because reason is also fully true

D. Cannot be determined

Ans: B

Assertion A is false: relative addressing is indeed used widely for branching, but the reason given (PC updated only during branches) is also incorrect, since the PC updates after every instruction, not just branches — making B the better characterization (A is incorrect as stated, since relative addressing CAN be used for data access too, not just branching).

Q147 | Conceptual · Moderate | Relative Addressing

The principal reason compilers use relative addressing for loop and conditional branch instructions is to:

A. Increase the absolute address range infinitely

B. Keep instruction encoding compact while supporting relocatable, efficient branching

C. Avoid using the Program Counter

D. Eliminate the need for an Instruction Register

Ans: B

Compilers favor relative addressing for branches/loops because it keeps instruction encoding compact (small displacement field) while still supporting code relocation and efficient jumps.

Q148 | Definition · Easy | Relative Addressing

Which of the following best defines Relative Addressing?

A. An addressing mode where the operand value is embedded in the instruction

B. An addressing mode where effective address = contents of an address field interpreted as offset from current PC

C. An addressing mode where the address field always equals the final operand address

D. An addressing mode requiring page tables for translation

Ans: B

This is the precise definition: the effective address equals the value in the address field interpreted as a signed offset added to the current PC.

Q149 | Scenario-based · Moderate | Relative Addressing

Scenario-based: A program is relocated to a different starting memory address. Instructions using relative addressing for internal branches will:

A. Need to be entirely rewritten with new absolute addresses

B. Continue to work correctly without modification, since offsets are relative to PC

C. Fail to execute at all

D. Only work if paging is disabled

Ans: B

Since the offsets are computed relative to the PC, not as absolute memory locations, the internal branch targets remain valid wherever the relocated code now resides — no changes to the offsets are needed.

Q150 | Conceptual · Moderate | Relative Addressing

Which addressing technique is typically combined with relative addressing in 'PC-relative' instruction set designs (e.g., for short jumps)?

A. Index register addition

B. Displacement added to current PC value

C. Two-level pointer indirection

D. Page table lookup

Ans: B

PC-relative addressing IS relative addressing applied specifically to instruction fetch/branch targets — both describe the same offset-from-PC mechanism.

Q151 | Conceptual · Easy | Relative Addressing

In relative addressing, what happens to the effective address if the displacement value is zero?

A. The effective address equals the current PC value

B. The effective address becomes zero

C. The instruction is skipped

D. The effective address becomes negative

Ans: A

If the displacement is zero, EA = PC + 0 = PC, i.e., the effective address simply equals the current PC value.

Q152 | Conceptual · Moderate | Relative Addressing

Which of the following is NOT a characteristic of Relative Addressing?

A. Effective address depends on PC

B. Useful for branch instructions

C. Supports position-independent code

D. Requires two full memory accesses to compute the address itself

Ans: D

Relative addressing depends on the PC, supports compact branch encoding, and enables position-independent code — but it requires only ONE memory reference to compute the address (addition, not memory access), so requiring two memory accesses is NOT one of its characteristics.

Q153 | Application-based · Moderate | Relative Addressing

Application-based: A conditional branch instruction is at address 500. After fetch, PC = 504. The displacement field contains -100. What is the branch target address?

A. 400

B. 404

C. 504

D. 604

Ans: B

Effective address = PC (504) + displacement (-100) = 404.

Q154 | Conceptual · Moderate | Relative Addressing

Which of the following is the most accurate reason relative addressing reduces the number of bits needed in the address/displacement field compared to direct addressing for nearby branch targets?

A. Because nearby targets can be expressed as small offsets from the current PC rather than full absolute addresses

B. Because relative addressing does not use any address field at all

C. Because relative addressing always targets address zero

D. Because the PC is never used in the calculation

Ans: A

Because nearby branch targets can be expressed as small signed offsets from the current instruction's PC, fewer bits are needed than would be required to store a full absolute address.

Q155 | Statement-based · Moderate | Relative Addressing

Statement-based: Consider the following about relative addressing:
(i) It is well suited to short, local branches within a subroutine.
(ii) The displacement is always unsigned.
(iii) It allows the same compiled code to run correctly after being relocated, as long as internal jumps remain relative.
Which are correct?

A. (i) and (iii) only

B. (i), (ii) and (iii)

C. (ii) and (iii) only

D. (i) and (ii) only

Ans: A

Local, short branches are a primary use case for relative addressing (i true), and PC-relative internal jumps continue to work correctly after relocation (iii true), but the displacement is signed, not unsigned, so (ii) is false.

Q156 | Assertion-Reason · Moderate | Relative Addressing

Assertion (A): A loop's backward branch (jumping back to the top of the loop) typically uses a negative displacement in relative addressing.
Reason (R): The loop's top is at a lower memory address than the branch instruction itself.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

A backward branch targets an earlier (lower) address than the current instruction, and reaching a lower address from the current PC requires adding a negative displacement — R correctly explains A.

Q157 | True-False MCQ · Moderate | Relative Addressing

True/False as MCQ: 'Relative addressing can only be used for branch instructions and never for accessing ordinary data operands.' This statement is:

A. True

B. False — some architectures also use PC-relative addressing for data, such as accessing nearby constants

C. True only in 8-bit microprocessors

D. True only when paging is enabled

Ans: B

While relative addressing is most associated with branches, some instruction sets also use PC-relative addressing to access nearby data (like literal pools or constants), so the blanket restriction to branches only is false.

Q158 | Comparison-based · Difficult | Relative Addressing

Comparison-based: For a branch target that is very far away in memory (beyond the displacement field's range), which approach is typically needed?

A. Relative addressing automatically extends its range

B. An absolute/direct jump instruction or a multi-instruction sequence is used instead

C. Indexed addressing must replace the PC

D. Paging is used to bypass the limitation

Ans: B

When a target lies beyond what the displacement field can represent, the architecture typically falls back to an absolute jump instruction or a longer instruction sequence, since the offset simply cannot be encoded.

Q159 | Scenario-based · Moderate | Relative Addressing

Scenario-based: Code compiled with relative addressing for internal branches is copied to a new location in memory without any changes to the offsets. Will the branches still work correctly?

A. No, the program will always crash

B. Yes, since each offset is computed relative to its own instruction's PC, not an absolute address

C. Only if the displacement field is in unsigned format

D. Only if direct addressing is also used

Ans: B

Since each branch's offset is computed relative to its own current PC rather than tied to an absolute address, relocating the entire block of code preserves the correctness of every internal relative branch.

Q160 | Application-based · Easy | Relative Addressing

Which of the following best illustrates an application-level analogy for relative addressing?

A. Giving someone a fixed street address

B. Giving someone directions like 'three blocks ahead from where you are now'

C. Giving someone a phone number that connects to another number

D. Giving someone a randomly generated code

Ans: B

Relative addressing computes targets as 'so many steps from here,' much like giving someone directions relative to their current position rather than a fixed street address.

Q161 | **Match-the-Following · Moderate** | **Relative Addressing**

Match the following:
List I: (1) Relative Addressing (2) Forward branch (3) Backward branch
List II: (P) Uses negative displacement (Q) EA = PC + displacement (R) Uses positive displacement
Choose the correct match:

A. 1-Q, 2-R, 3-P

B. 1-P, 2-Q, 3-R

C. 1-Q, 2-P, 3-R

D. 1-R, 2-Q, 3-P

Ans: A

Relative Addressing computes the effective address as EA = PC + displacement; a forward branch (jumping to a later address) uses a positive displacement, while a backward branch (jumping to an earlier address) uses a negative displacement — matching 1-Q, 2-R, 3-P.

Section 5: Paging

Q162 | **Definition · Easy** | **Paging**

Paging is primarily a memory management technique used to:

A. Encrypt memory contents

B. Divide memory into fixed-size blocks (pages/frames) to manage logical-to-physical address translation

C. Increase CPU clock speed

D. Compress program code

Ans: B

Paging is a memory management scheme that divides both logical and physical memory into equal, fixed-size blocks so that any free block can be used, simplifying address translation.

Q163 | **Definition · Easy** | **Paging**

In Paging, the logical (virtual) address space is divided into fixed-size blocks called:

A. Frames

B. Pages

C. Segments

D. Sectors

Ans: B

In paging terminology, the fixed-size blocks of the logical (virtual) address space are specifically called pages.

Q164 | **Definition · Easy** | **Paging**

In Paging, the corresponding fixed-size blocks of physical memory are called:

A. Pages

B. Frames

C. Records

D. Blocks of cache

Ans: B

The corresponding fixed-size blocks within physical memory are called frames; pages map onto frames during translation.

Q165 | Definition · Easy | Paging

Address translation in Paging is performed using a data structure called the:

A. Segment table

B. Page table

C. File allocation table

D. Interrupt vector table

Ans: B — The page table is the data structure (maintained per process) that records which frame in physical memory each logical page is currently mapped to.

Q166 | Conceptual · Easy | Paging

In Paging, a logical address is typically divided into which two parts?

A. Base and index

B. Page number and page offset (displacement within the page)

C. Opcode and operand

D. Segment number and limit

Ans: B — A logical address under paging is split into a page number (which page) and a page offset/ displacement (which byte within that page) — these two fields together locate any byte.

Q167 | Conceptual · Moderate | Paging

The page offset (displacement) in a paged address scheme is used to:

A. Select which page table to use

B. Specify the exact location of data within a page/frame, and is added unchanged during translation

C. Determine the frame number directly

D. Select the CPU mode

Ans: B — The offset identifies the exact byte within a page/frame and is carried through unchanged into the physical address, since both the page and its corresponding frame are the same size.

Q168 | Conceptual · Moderate | Paging

A key advantage of Paging is:

A. It eliminates external fragmentation by allocating memory in fixed-size frames

B. It always speeds up CPU clock rate

C. It requires no address translation

D. It eliminates the need for main memory

Ans: A — Because all frames are the same fixed size, any free frame can hold any page, so there's no need for contiguous blocks of varying sizes — this directly eliminates external fragmentation.

Q169 | Conceptual · Moderate | Paging

A disadvantage commonly associated with Paging is:

A. External fragmentation

B. Internal fragmentation, since a process may not fully use the last allocated page/frame

C. Inability to support virtual memory

D. Requirement of contiguous memory allocation

Ans: B

Since a process's memory requirement rarely divides evenly into whole pages, the last allocated page is often only partially used, wasting space within it — this is internal fragmentation.

Q170 | Application-based · Moderate | Paging

Application-based: If the page size is 4 KB and a process requires 10 KB of memory, how many pages will be allocated (assuming whole pages only)?

A. 2

B. 3

C. 4

D. 10

Ans: B

10 KB needs ceil(10/4) = 3 pages of 4 KB each (the 3rd page is only partially filled, illustrating internal fragmentation).

Q171 | Conceptual · Moderate | Paging

In Paging, the Page Table Base Register (PTBR) is used to:

A. Store the page offset

B. Point to the location of the current process's page table in memory

C. Store the frame size

D. Store the value of the accumulator

Ans: B

The PTBR stores the base (starting) memory address of the currently running process's page table, so the MMU knows where to look up page-to-frame mappings.

Q172 | Statement-based · Moderate | Paging

Statement-based: Consider the following statements about Paging:
(i) Pages and frames are of the same fixed size.
(ii) The page table maps page numbers to frame numbers.
(iii) Paging causes external fragmentation.
Which statements are correct?

A. (i) and (ii) only

B. (ii) and (iii) only

C. (i) and (iii) only

D. All three

Ans: A

Statement (iii) is false because paging eliminates external fragmentation (not causes it); only statements (i) and (ii), describing equal page/frame sizes and the page table's mapping role, are correct.

Q173 | Assertion-Reason · Moderate | Paging

Assertion (A): Paging helps eliminate external fragmentation in memory allocation.
Reason (R): Memory is divided into fixed-size frames, so any free frame can hold any page, removing the need for contiguous allocation.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Because every frame is the same fixed size, free frames can be allocated to any page without needing contiguous space, which is exactly why external fragmentation does not occur — R correctly explains A.

Q174 | True-False MCQ · Easy | Paging

True/False as MCQ: 'In paging, pages and frames must be exactly the same size for correct address mapping.' This statement is:

A. True

B. False

C. True only in segmented systems

D. True only for virtual memory disabled systems

Ans: A

For the page-to-frame translation to work correctly (offset carried over unchanged), pages and frames must be defined with identical fixed sizes.

Q175 Conceptual · Moderate — Paging

In address translation using paging, given a logical address with page number p and offset d, the physical address is formed as:

A. Frame number (from page table entry for p) concatenated/combined with offset d

B. p multiplied by d

C. p added to the base register only

D. d alone, ignoring p

Ans: A The physical address is formed by taking the frame number obtained from the page table (using page number p as the index) and appending the unchanged offset d to it.

Q176 Conceptual · Moderate — Paging

Which hardware component is used to speed up page table lookups by caching recent translations?

A. Cache memory only

B. Translation Look-aside Buffer (TLB)

C. Memory Buffer Register

D. Stack pointer

Ans: B A TLB is a small, fast cache of recent page-table lookups, used so the CPU doesn't have to access the full page table in memory for every single memory reference.

Q177 Conceptual · Easy — Paging

Paging is a key mechanism that enables which broader memory management concept?

A. Direct memory access (DMA)

B. Virtual memory

C. Cache coherence

D. Instruction pipelining

Ans: B Paging is the underlying mechanism that allows a process's logical address space to be mapped onto possibly scattered physical frames, which is what makes virtual memory possible.

Q178 Scenario-based · Difficult — Paging

Scenario-based: A system uses a page size of 1 KB. A logical address is 2570 (decimal). Which page number does this address belong to?

A. Page 1

B. Page 2

C. Page 3

D. Page 25

Ans: B With a 1 KB (1024-byte) page size, address 2570 falls in page number floor(2570/1024) = 2 (page 2 covers addresses 2048–3071).

Q179 | Comparison-based · Moderate | Paging

Comparison-based: Unlike contiguous memory allocation, paging allocates memory to a process:

A. In one large contiguous block only

B. In fixed-size, possibly non-contiguous frames scattered in physical memory

C. Only at the start of physical memory

D. Only in secondary storage

Ans: B

Unlike contiguous allocation (which needs one unbroken block per process), paging splits a process's memory needs across fixed-size frames that may be scattered anywhere in physical memory.

Q180 | Definition · Moderate | Paging

Which of the following best describes 'internal fragmentation' in the context of paging?

A. Wasted space outside allocated memory blocks

B. Wasted space within the last allocated page/frame when the process does not fully use it

C. Fragmentation of the hard disk

D. Fragmentation of the CPU cache only

Ans: B

Internal fragmentation specifically refers to the wasted space inside the last page/frame allocated to a process when that page isn't fully used by the process's data.

Q181 | Conceptual · Moderate | Paging

Each entry in a page table typically contains (at minimum):

A. The corresponding frame number

B. The full physical address of every byte

C. The entire program code

D. The CPU's clock speed

Ans: A

At minimum, each page table entry must record the frame number that the corresponding page is mapped to, so the CPU can translate logical to physical addresses.

Q182 | Application-based · Moderate | Paging

Which of these is an application/use-case benefit of paging?

A. Simplified memory allocation and support for non-contiguous physical memory usage, aiding virtual memory implementation

B. Eliminating the need for a page table

C. Removing the need for an operating system

D. Increasing instruction execution speed directly

Ans: A

Paging's core benefit is allowing non-contiguous physical memory allocation, which removes the need to find one large contiguous block and underlies virtual memory implementations.

Q183 | **Application-based · Difficult** | **Paging**

Application-based: With a page size of 2 KB, logical address 5000 has which page number and offset (page numbers starting from 0)?

A. Page 2, Offset 904

B. Page 2, Offset 1000

C. Page 3, Offset 0

D. Page 1, Offset 5000

Ans: A

With a 2 KB (2048-byte) page size, address 5000: page number = floor(5000/2048) = 2, offset = 5000 - (2*2048) = 5000 - 4096 = 904.

Q184 | **Assertion-Reason · Moderate** | **Paging**

Assertion (A): A Translation Look-aside Buffer (TLB) improves performance of paging-based systems.
Reason (R): It avoids the need to access the page table in main memory for every address translation by caching recent page-to-frame mappings.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

A TLB caches recent page-to-frame translations so most memory references avoid the extra page-table lookup, directly improving the performance of paging-based systems — R correctly explains A.

Q185 | **Comparison-based · Moderate** | **Paging**

Which statement correctly distinguishes Paging from Segmentation?

A. Paging uses fixed-size blocks; Segmentation uses variable-size logical units

B. Paging uses variable-size blocks; Segmentation uses fixed-size blocks

C. Both use exactly the same block sizes

D. Neither requires address translation

Ans: A

Paging splits memory into fixed-size pages/frames, while segmentation divides a program into logically meaningful but variable-size segments (like code, data, stack) — this is the defining contrast.

Q186 Match-the-Following · Moderate — Paging

Match the following:
List I: (1) Page (2) Frame (3) Page Table (4) Offset
List II: (P) Maps pages to frames (Q) Fixed-size block of physical memory (R) Fixed-size block of logical memory (S) Location within a page
Choose correct match:

A. 1-R, 2-Q, 3-P, 4-S

B. 1-Q, 2-R, 3-P, 4-S

C. 1-R, 2-P, 3-Q, 4-S

D. 1-S, 2-Q, 3-P, 4-R

Ans: A

Page (1) is the fixed logical block (R), Frame (2) is the fixed physical block (Q), Page Table (3) maps pages to frames (P), and Offset (4) locates a byte within a page (S) — matching 1-R, 2-Q, 3-P, 4-S.

Q187 Conceptual · Easy — Paging

In Paging, if a page is not currently present in physical memory when referenced, this event is called:

A. Page fault

B. Page hit

C. Segmentation fault only

D. Bus error

Ans: A

A page fault occurs precisely when a referenced page is not currently loaded into a physical frame, requiring the OS to fetch it from secondary storage.

Q188 Conceptual · Difficult — Paging

Which of the following is generally considered an advantage of using small page sizes?

A. Reduced internal fragmentation, since less space is wasted per page

B. Larger page tables requiring less memory

C. Always faster TLB lookups regardless of size

D. Elimination of page faults entirely

Ans: A

Smaller pages mean the last (partially used) page wastes less unused space, directly reducing internal fragmentation.

Q189 | Conceptual · Difficult | Paging

Which of the following is generally considered a disadvantage of using very small page sizes?

A. Increased internal fragmentation

B. Larger page tables, requiring more memory overhead to store mapping information

C. Elimination of the page table

D. Reduced number of pages per process

Ans: B

Smaller pages mean more pages per process, which requires more page table entries and hence a larger page table, increasing memory overhead for the mapping structure itself.

Q190 | Application-based · Moderate | Paging

Paging primarily supports which advanced capability in modern operating systems?

A. Running a process whose logical address space exceeds available contiguous physical memory, using non-contiguous allocation

B. Direct execution of instructions without fetch/decode/execute

C. Elimination of all addressing modes

D. Replacing the CPU's control unit

Ans: A

By allowing a process's pages to be scattered across available physical frames (not requiring one contiguous block), paging lets a process's address space exceed what would otherwise fit contiguously, enabling virtual memory.

Q191 | Application-based · Moderate | Paging

Application-based: A system uses 32-bit logical addresses with a page size of 4 KB (2^12 bytes). How many bits are used for the page offset?

A. 8 bits

B. 10 bits

C. 12 bits

D. 16 bits

Ans: C

A 4 KB page size is 2^12 bytes, so 12 bits are needed to address every byte within a single page (the offset).

Q192 | Application-based · Difficult | Paging

Following from the previous setup (32-bit address, 4 KB pages), how many bits remain for the page number?

A. 12 bits

B. 16 bits

C. 20 bits

D. 24 bits

Ans: C

With 32-bit total addresses and 12 bits used for the offset, the remaining 32 - 12 = 20 bits are used for the page number.

Q193 | Conceptual · Moderate | Paging

Which of the following is a correct statement about the relationship between page size and the number of page table entries needed for a fixed-size address space?

A. Larger page size means more page table entries are needed

B. Larger page size means fewer page table entries are needed, since each entry covers more memory

C. Page size has no relationship with the number of page table entries

D. Page table entries are unrelated to page size

Ans: B

A larger page size means each page table entry covers more memory, so fewer entries (and thus a smaller page table) are needed to map the same total address space.

Q194 | Assertion-Reason · Difficult | Paging

Assertion (A): Multi-level (hierarchical) page tables are used in systems with very large address spaces.
Reason (R): A single-level page table for a large address space would itself require an impractically large contiguous block of memory.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

A single flat page table for a huge address space would itself need a massive contiguous memory allocation, which hierarchical (multi-level) page tables avoid by breaking the table into smaller, manageable pieces — R correctly explains A.

Q195 | Statement-based · Moderate | Paging

Statement-based: Consider the following about paging:
(i) A page fault always indicates a programming error.
(ii) Paging allows processes to use more logical memory than is physically available, when combined with secondary storage.
(iii) The page table is consulted on every memory reference for address translation.
Which are correct?

A. (ii) and (iii) only

B. (i) and (ii) only

C. (i) and (iii) only

D. All three

Ans: A

A page fault simply means the needed page isn't currently in a physical frame, not necessarily a programming error (i false); paging combined with secondary storage does let logical memory exceed physical memory (ii true); and the page table (or TLB) is indeed consulted for every reference (iii true) — so (ii) and (iii) are correct.

Q196 | True-False MCQ · Moderate | Paging

True/False as MCQ: 'A Translation Look-aside Buffer (TLB) miss means the page is definitely not in physical memory.' This statement is:

A. True

B. False — a TLB miss only means the translation wasn't cached; the page table may still show it is resident in memory

C. True only for direct-mapped TLBs

D. True only when paging is disabled

Ans: B

A TLB miss only means the translation wasn't found in the fast cache; the operating system must still check the full page table, where the page may well be marked as present in memory — so the statement is false.

Q197 | Conceptual · Moderate | Paging

Which of the following best explains why frame size in physical memory must equal page size in logical memory?

A. So that any page can be mapped onto any free frame without size-mismatch issues

B. Because frames and pages are unrelated concepts

C. Because the CPU clock requires it

D. Because of ALU constraints

Ans: A

Identical page and frame sizes ensure that the offset portion of an address can be carried over unchanged between logical and physical addresses, and any free frame can hold any page without size mismatches.

Q198 | Scenario-based · Difficult | Paging

Scenario-based: A process needs 18 KB of memory, and the system page size is 4 KB. How many pages are allocated, and how much internal fragmentation results in the last page?

A. 4 pages, 2 KB fragmentation

B. 5 pages, 2 KB fragmentation

C. 5 pages, 4 KB fragmentation

D. 4 pages, 4 KB fragmentation

Ans: B

18 KB / 4 KB per page = 4.5, rounded up to 5 pages; the 5th page holds only 2 KB of actual data, wasting 4 KB - 2 KB = 2 KB as internal fragmentation.

Q199 | Comparison-based · Moderate | Paging

Comparison-based: Which causes more internal fragmentation on average: a larger page size or a smaller page size?

A. A larger page size, since the unused portion of the last page can be bigger

B. A smaller page size always causes more fragmentation

C. Page size has no effect on internal fragmentation

D. Internal fragmentation only depends on frame count

Ans: A

A larger page size means the unused portion of a partially filled last page can itself be larger, so on average larger pages tend to cause more internal fragmentation than smaller ones.

Q200 | Match-the-Following · Moderate | Paging

Match the following:
List I: (1) Page Table (2) TLB (3) Page Fault (4) Frame
List II: (P) Fixed-size block of physical memory (Q) Cache of recent translations (R) Maps pages to frames (S) Occurs when a needed page isn't in memory
Choose the correct match:

A. 1-R, 2-Q, 3-S, 4-P

B. 1-Q, 2-R, 3-S, 4-P

C. 1-R, 2-S, 3-Q, 4-P

D. 1-P, 2-Q, 3-R, 4-S

Ans: A

The Page Table (1) maps pages to frames (R), the TLB (2) caches recent translations (Q), a Page Fault (3) occurs when a page isn't resident (S), and a Frame (4) is the fixed physical block (P) — matching 1-R, 2-Q, 3-S, 4-P.

Q201 | **Conceptual · Moderate** | **Paging**

Which of the following is an accurate statement about the cost of a page fault compared to a TLB hit?

A. A page fault is much more expensive, since it may require a disk access to bring in the page

B. A page fault is always faster than a TLB hit

C. They cost exactly the same

D. A page fault never requires disk access

Ans: A

A page fault is significantly more expensive than a TLB hit because it may require a slow disk access to load the missing page into memory, unlike a TLB hit which resolves in essentially one cache lookup.

Section 6: Indexed Addressing

Q202 | **Definition · Easy** | **Indexed Addressing**

In Indexed Addressing, the effective address is computed as:

A. Contents of address field only

B. Contents of an Index Register plus the address (or displacement) given in the instruction

C. PC plus displacement

D. Contents of memory pointed to by address field

Ans: B

Indexed Addressing computes the effective address by adding the contents of an index register to a base address or displacement specified in the instruction: EA = Base/Displacement + Index Register.

Q203 | **Conceptual · Easy** | **Indexed Addressing**

Indexed Addressing is primarily used to facilitate efficient access to:

A. A single fixed scalar variable

B. Elements of arrays and tables, especially within loops

C. Branch target addresses only

D. Page table entries directly

Ans: B

By letting the index register's value change, the same instruction can access successive elements of an array or table — this is the primary motivation for indexed addressing.

Q204 | **Definition · Easy** | **Indexed Addressing**

Which register holds the value that is added to the base/displacement address in Indexed Addressing?

A. Program Counter

B. Stack Pointer

C. Index Register

D. Instruction Register

Ans: C

The index register is the dedicated register whose value is combined with the base/displacement to form the effective address in indexed addressing.

Q205 | Conceptual · Moderate | Indexed Addressing

A key advantage of Indexed Addressing is:

A. It cannot be used in loops

B. It allows easy and efficient traversal of array elements by simply changing the index register's value

C. It requires no registers at all

D. It always uses absolute addressing exclusively

Ans: B

Since only the index register's value needs to change (not the instruction itself) to move from one array element to the next, traversal becomes simple and efficient, especially in loops.

Q206 | Application-based · Moderate | Indexed Addressing

Application-based: An array starts at base address 2000. Using Indexed Addressing with an index register value of 5 (and each element occupying 1 byte), what is the effective address of the 6th element (index 5)?

A. 2000

B. 2005

C. 2010

D. 2055

Ans: B

Base 2000 + Index register value 5 (1 byte per element) gives EA = 2000 + 5 = 2005, which is the address of the 6th element (index 5).

Q207 | Statement-based · Moderate | Indexed Addressing

Statement-based: Consider the following about Indexed Addressing:
(i) The effective address depends on the index register's content.
(ii) It is particularly efficient for array/loop processing.
(iii) It always requires two memory accesses to fetch the operand.
Which are correct?

A. (i) and (ii) only

B. (ii) and (iii) only

C. (i) and (iii) only

D. All three

Ans: A

Statement (iii) is false — indexed addressing typically requires only one memory access for the operand itself (the addition of base and index is done in registers, not via an extra memory reference); statements (i) and (ii) are correct.

Q208 | Assertion-Reason · Moderate | Indexed Addressing

Assertion (A): Indexed addressing is well-suited for processing array elements in a loop.
Reason (R): The index register can be incremented each iteration to point to successive array elements without modifying the instruction itself.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Because the index register can simply be incremented each loop iteration to point to the next array element without changing the instruction, indexed addressing is naturally suited to array/loop processing — R correctly explains A.

Q209 | True-False MCQ · Easy | Indexed Addressing

True/False as MCQ: 'In indexed addressing, the index register's value remains constant throughout a program and cannot be modified.' This statement is:

A. True

B. False, since the index register is typically updated to access different elements

C. True only in 8-bit systems

D. True only for read-only arrays

Ans: B

In a loop traversing an array, the index register must be updated (incremented/decremented) each iteration to access different elements — so it does NOT remain constant.

Q210 | Conceptual · Moderate | Indexed Addressing

Which of the following is a disadvantage of Indexed Addressing?

A. It cannot represent any offset

B. Additional hardware/cycles may be needed to compute the address by adding index register content to the base/displacement

C. It eliminates the need for arrays

D. It is identical in speed to direct addressing always

Ans: B

Although flexible, indexed addressing needs extra hardware (an adder) and an extra cycle to compute base + index before the memory access can occur, adding some overhead compared to simpler modes.

Q211 | Scenario-based · Difficult | Indexed Addressing

Scenario-based: A program needs to access the 10th element of an array stored from address 500, with each element 4 bytes wide. Which addressing mode is most appropriate, and what would the index register hold?

A. Direct Addressing; index register holds 500

B. Indexed Addressing; index register holds 36 (9*4)

C. Relative Addressing; index register holds 10

D. Indirect Addressing; index register holds 4

Ans: B

To access the 10th element (0-based index 9) with 4-byte elements, the index register should hold 9*4 = 36, which is added to the base address — this is the classic use of indexed addressing for array access.

Q212 | Comparison-based · Moderate | Indexed Addressing

Indexed Addressing differs from Relative Addressing mainly in that:

A. Indexed addressing uses an index register, while relative addressing uses the Program Counter as the reference for computing the effective address

B. Both use the Program Counter

C. Both use only the address field with no addition

D. Indexed addressing requires two memory accesses always

Ans: A

Indexed addressing forms EA using an index register added to a base, while relative addressing forms EA using the Program Counter added to a displacement — the reference point (index register vs PC) is the key distinguishing factor.

Q213 | Comparison-based · Moderate | Indexed Addressing

Comparison-based: Compared to Direct Addressing, Indexed Addressing provides:

A. No additional flexibility

B. Greater flexibility for accessing sequential/array data by varying the index register, at the cost of an extra addition operation

C. Smaller address range always

D. Slower decode phase only, with no benefit

Ans: B

Direct addressing always points to one fixed location, while indexed addressing can reach a whole sequence of locations just by varying the index register — at the cost of needing an addition operation each time.

Q214 | Match-the-Following · Moderate | Indexed Addressing

Match the following:
List I: (1) Indexed Addressing (2) Index Register (3) Array traversal
List II: (P) Common application of indexed addressing (Q) Added to base address to get EA (R) EA = Base + Index
Choose correct match:

A. 1-R, 2-Q, 3-P

B. 1-Q, 2-R, 3-P

C. 1-P, 2-Q, 3-R

D. 1-R, 2-P, 3-Q

Ans: A

Indexed Addressing (1) computes EA = Base + Index (R), the Index Register (2) is added to the base address (Q), and Array traversal (3) is the typical application (P) — matching 1-R, 2-Q, 3-P.

Q215 | Application-based · Easy | Indexed Addressing

In Indexed Addressing, when used inside a loop to traverse an array, the index register is typically:

A. Reset to zero every iteration without increment

B. Incremented (or decremented) each iteration to point to the next array element

C. Never changed throughout execution

D. Used only once, at compile time

Ans: B

To move through successive array elements in a loop, the index register is updated (typically incremented) each pass so the same instruction accesses the next element.

Q216 | Conceptual · Moderate | Indexed Addressing

Which of the following best defines the role of the base address (or displacement) in Indexed Addressing?

A. It denotes the starting address of the data structure (e.g., array), to which the index register's value is added

B. It is multiplied by the index value

C. It is ignored when index is non-zero

D. It always denotes the Program Counter

Ans: A

The base/displacement value marks where the data structure (e.g., array) begins in memory; the index register's value is then added to it to reach a specific element within that structure.

Q217 | Application-based · Easy | Indexed Addressing

Indexed Addressing is most beneficial when a program needs to:

A. Access a single fixed memory location repeatedly

B. Access multiple, sequential locations such as elements of an array or table efficiently

C. Avoid using any registers

D. Avoid branch instructions

Ans: B

Indexed addressing shines when a program must step through multiple, sequential memory locations (like array elements), since only the index register needs to change between accesses.

Q218 | Assertion-Reason · Moderate | Indexed Addressing

Assertion (A): Indexed addressing requires hardware support for addition (an adder) to compute the effective address.
Reason (R): The effective address is computed by adding the index register's value to a base/displacement value from the instruction.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Computing EA = base + index requires an adder in the addressing hardware, and indeed this addition is necessary every time an indexed-addressing instruction executes — R correctly explains A.

Q219 | Conceptual · Moderate | Indexed Addressing

Which of the following is NOT a characteristic of Indexed Addressing?

A. Uses an index register

B. Effective address = base/displacement + index

C. Particularly suited for array processing

D. Effective address is always equal to PC value

Ans: D

Indexed addressing always involves the index register added to a base/displacement; the effective address being 'always equal to PC' describes relative addressing instead, not indexed addressing — so that statement is NOT a characteristic of indexed addressing.

Q220 | **Application-based · Easy** | **Indexed Addressing**

Application-based: If the base address in an indexed addressing instruction is 1000 and the index register currently holds 25, what is the effective address?

A. 975

B. 1000

C. 1025

D. 25000

Ans: C EA = base (1000) + index register (25) = 1025.

Q221 | **Conceptual · Easy** | **Indexed Addressing**

Indexed Addressing is conceptually similar to which high-level programming construct?

A. A constant literal

B. Array subscripting/indexing (e.g., arr[i])

C. A function pointer dereference

D. A page table lookup

Ans: B Array subscripting (e.g., arr[i]) directly mirrors indexed addressing: a base location (the array) plus an offset (the index) determines the exact element accessed.

Q222 | **Comparison-based · Moderate** | **Indexed Addressing**

Which of the following correctly distinguishes Indexed from Indirect Addressing?

A. Indexed addressing adds an index register value to a base address; Indirect addressing uses the address field to point to another memory location holding the actual address

B. Both retrieve the operand's address from the same memory location

C. Indexed addressing never uses any register

D. Indirect addressing always uses an index register

Ans: A Indexed addressing adds an index register's value to a base address to compute EA directly; indirect addressing instead uses the address field to point to ANOTHER memory location that holds the real address — the mechanisms are fundamentally different.

Q223 | Application-based · Difficult | Indexed Addressing

Scenario-based: A table of 100 elements is stored starting at address 4096, with each element occupying 2 bytes. Using Indexed Addressing, what value should the index register hold to access the 50th element (0-based indexing, i.e., element at index 49)?

A. 49

B. 50

C. 98

D. 100

Ans: C

For 2-byte elements, the 50th element (index 49) is at offset 49*2 = 98 bytes from the base, so the index register should hold 98.

Q224 | Conceptual · Moderate | Indexed Addressing

Which of the following best describes an advantage of using Indexed Addressing over repeatedly modifying instructions with new direct addresses?

A. It avoids the need to alter the instruction's opcode/address field for each array element; only the index register is updated

B. It eliminates the need for any addressing mode

C. It requires recompiling the program for each new index

D. It cannot be used with loops

Ans: A

Since only the index register changes between accesses (rather than rewriting the instruction's address field for every new element), indexed addressing avoids the need to modify code for each array access.

Q225 | True-False MCQ · Easy | Indexed Addressing

True/False as MCQ: 'Indexed addressing effective address calculation requires an addition operation between the index register and a base/displacement value.' This statement is:

A. True

B. False

C. True only for 16-bit processors

D. True only when index register is zero

Ans: A

Indexed addressing requires only one addition (base + index register) to form EA, by definition — this is unambiguously true.

Q226 | **Application-based · Easy** | **Indexed Addressing**

Which of the following is an example of a real-world application best supported by Indexed Addressing?

A. Reading a single global configuration flag

B. Iterating through and summing all elements of a large numeric array

C. Branching to a fixed subroutine address

D. Reading the value of the Program Counter

Ans: B

Iterating through and summing array elements is a canonical real-world use of indexed addressing, since the index register can be incremented each step to reach the next value.

Q227 | **Conceptual · Difficult** | **Indexed Addressing**

In some architectures, more than one index register may be used to support:

A. Single scalar access only

B. Multi-dimensional array access by combining offsets from multiple index registers

C. Eliminating the need for a base address

D. Disabling the addressing hardware

Ans: B

Some architectures use multiple index registers so that offsets from each (e.g., row and column indices) can be combined to address elements of multi-dimensional arrays.

Q228 | **Application-based · Difficult** | **Indexed Addressing**

Application-based: A 2D array stored in row-major order has a base address of 3000. Each row has 10 elements of 4 bytes each. Using indexed addressing concepts, what offset (in bytes) corresponds to the start of row 3 (0-indexed)?

A. 30

B. 40

C. 120

D. 3000

Ans: C

Row 3 (0-indexed) starts after rows 0, 1, and 2, each containing 10 elements of 4 bytes = 40 bytes per row; so the offset to row 3 is 3 × 40 = 120 bytes.

Q229 | Conceptual · Moderate | Indexed Addressing

Which of the following is the most accurate reason indexed addressing is preferred over repeatedly hardcoding new direct addresses when processing a list?

A. Because indexed addressing eliminates the need for any base address

B. Because only the index register needs updating, leaving the instruction's encoded base/displacement unchanged

C. Because indexed addressing does not require any addition

D. Because the index register is the same as the Program Counter

Ans: B

The key efficiency of indexed addressing is that only the index register's value needs to change between accesses, leaving the instruction's base/displacement field untouched.

Q230 | Statement-based · Moderate | Indexed Addressing

Statement-based: Consider the following about indexed addressing:
(i) Some architectures support auto-increment/auto-decrement of the index register as part of the addressing mode.
(ii) The index register's width limits the range of offsets it can represent.
(iii) Indexed addressing is unrelated to array processing.
Which are correct?

A. (i) and (ii) only

B. (ii) and (iii) only

C. (i) and (iii) only

D. All three

Ans: A

Auto-increment/decrement of the index register is a real hardware feature in many architectures (i true), and the index register's bit-width does cap the representable offset range (ii true), but indexed addressing is in fact closely tied to array processing, so (iii) is false.

Q231 | Assertion-Reason · Difficult | Indexed Addressing

Assertion (A): Indexed addressing can be combined with indirect addressing in some instruction sets (indexed-indirect or indirect-indexed modes).
Reason (R): Combining modes allows more complex data structures (e.g., arrays of pointers) to be accessed efficiently.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Combining indexed and indirect addressing allows accessing, for example, the address stored in the i-th slot of an array of pointers — useful for complex structures, which is exactly what R explains about A.

Q232 | True-False MCQ · Moderate | Indexed Addressing

True/False as MCQ: 'Auto-increment indexed addressing automatically updates the index register after each access, without a separate increment instruction.' This statement is:

A. True

B. False

C. True only for floating-point arrays

D. True only for direct addressing

Ans: A

Auto-increment indexed addressing is a real hardware feature where the index register's value is automatically adjusted (incremented/decremented) by the addressing hardware itself after each use, without needing a separate explicit increment instruction.

Q233 | Application-based · Moderate | Indexed Addressing

Which of the following best explains why indexed addressing is commonly used in implementing string operations (e.g., copying a string character by character)?

A. Because each character can be accessed by incrementing the index register relative to the string's base address

B. Because strings cannot be stored in memory otherwise

C. Because indexed addressing avoids the need for any base address

D. Because strings always have a fixed length of one

Ans: A

Incrementing the index register by one each time naturally lets the same instruction step through each character of a string relative to its starting (base) address.

Q234 | Comparison-based · Moderate | Indexed Addressing

Comparison-based: Which addressing mode would be more efficient for accessing every 5th element of a large array in a single pass: Direct Addressing (rewriting the address each time) or Indexed Addressing (incrementing the index by 5 each time)?

A. Direct Addressing, since it always uses one memory access

B. Indexed Addressing, since only the index register needs updating without modifying the instruction

C. Both are exactly equally efficient

D. Neither addressing mode can perform this access pattern

Ans: B

With indexed addressing, only the index register's value (incremented by 5 each pass) needs to change; the instruction itself stays the same, avoiding the overhead of rewriting a direct address every time.

Q235 | **Scenario-based · Moderate** | **Indexed Addressing**

Scenario-based: A matrix multiplication routine needs to access elements using both a row offset and a column offset. Which addressing approach is most natural?

A. Using a single fixed direct address for the whole matrix

B. Using indexed addressing with the index register computed from row and column positions relative to the matrix's base address

C. Using only the Program Counter

D. Using paging exclusively, with no addressing mode

Ans: B

Combining a base address with offsets derived from row and column positions (computed into the index register) is the natural way indexed addressing supports multi-dimensional array access such as in matrix operations.

Q236 | **Match-the-Following · Moderate** | **Indexed Addressing**

Match the following:
List I: (1) Indexed Addressing (2) Auto-increment (3) Base address
List II: (P) Starting location of a data structure (Q) Automatically updates index after access (R) EA = Base + Index
Choose the correct match:

A. 1-R, 2-Q, 3-P

B. 1-Q, 2-R, 3-P

C. 1-P, 2-Q, 3-R

D. 1-R, 2-P, 3-Q

Ans: A

Indexed Addressing (1) computes EA = Base + Index (R), Auto-increment (2) automatically updates the index after access (Q), and the Base address (3) marks where the data structure begins (P) — matching 1-R, 2-Q, 3-P.

Section 7: Comparative & Mixed Addressing Techniques

Q237 | **Comparison-based · Easy** | **Comparative & Mixed Addressing Techniques**

Comparison-based: Which addressing mode provides the FASTEST operand access (fewest memory references) among Direct and Indirect Addressing?

A. Direct Addressing (one memory reference)

B. Indirect Addressing (two memory references)

C. Both are equally fast

D. Cannot be determined

Ans: A

Direct addressing needs only one memory reference to retrieve the operand, fewer than indirect addressing's two, making it the fastest of these two options.

Q238 | **Conceptual · Easy** | **Comparative & Mixed Addressing Techniques**

Which addressing technique computes the effective address relative to the current Program Counter value?

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Paging

Ans: C

Relative addressing explicitly bases its effective-address calculation on the current Program Counter value plus a displacement — this PC-dependency is its defining trait.

Q239 | **Conceptual · Easy** | **Comparative & Mixed Addressing Techniques**

Which addressing technique uses an Index Register added to a base/displacement value to compute the effective address?

A. Direct Addressing

B. Indexed Addressing

C. Indirect Addressing

D. Paging

Ans: B

Indexed addressing's defining computation is adding an index register's value to a base/displacement to obtain the effective address.

Q240 | Conceptual · Moderate | Comparative & Mixed Addressing Techniques

Which of the following is a memory management technique (rather than an instruction addressing mode in the classical CPU sense) among the listed topics?

A. Direct Addressing

B. Indirect Addressing

C. Paging

D. Relative Addressing

Ans: C

Paging is a memory management/address-translation scheme implemented largely in hardware (MMU) and the OS, mapping logical pages to physical frames — unlike Direct, Indirect, and Relative, which are instruction-level addressing modes.

Q241 | Statement-based · Moderate | Comparative & Mixed Addressing Techniques

Statement-based: Consider the following:
(i) Direct addressing offers the smallest address range relative to its field size, but fastest access.
(ii) Indirect addressing offers a larger address range but slower access.
(iii) Relative addressing computes addresses relative to PC, supporting relocatable code.
Which statements are correct?

A. (i) and (ii) only

B. (i), (ii) and (iii)

C. (ii) and (iii) only

D. (i) and (iii) only

Ans: B

Statement (i) and (ii) correctly describe direct and indirect addressing's speed/range trade-off, and (iii) correctly notes relative addressing's PC-relative, relocatable nature — all three are accurate.

Q242 | Match-the-Following · Moderate | Comparative & Mixed Addressing Techniques

Match the following addressing techniques with their defining feature:
List I: (1) Direct (2) Indirect (3) Relative (4) Indexed
List II: (P) EA = PC + Offset (Q) EA = Base + Index Register (R) EA = Address field value (S) EA = Contents of address pointed to by address field
Choose correct match:

A. 1-R, 2-S, 3-P, 4-Q

B. 1-S, 2-R, 3-Q, 4-P

C. 1-R, 2-Q, 3-S, 4-P

D. 1-P, 2-Q, 3-R, 4-S

Ans: A

Direct (1) gives EA directly from the address field (R), Indirect (2) gives the address of a memory location holding EA (S), Relative (3) computes EA as PC + Offset (P), and Indexed (4) computes EA as Base + Index Register (Q) — matching 1-R, 2-S, 3-P, 4-Q.

Q243 | Assertion-Reason · Difficult | Comparative & Mixed Addressing Techniques

Assertion (A): Paging and Indexed Addressing both involve adding an offset/displacement to obtain a final address.
Reason (R): In Paging, the page offset is combined with the frame number; in Indexed Addressing, the index register value is added to a base address.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

In paging, the offset is combined with the translated frame number, and in indexed addressing, the index register's value is added to a base — both genuinely involve combining an offset/displacement with a base reference to form the final address, so R correctly explains A.

Q244 | Comparison-based · Moderate | Comparative & Mixed Addressing Techniques

Which addressing modes among the listed topics require exactly ONE memory access (besides the instruction fetch itself) to retrieve the operand, in their simplest form?

A. Direct and Relative Addressing

B. Indirect Addressing only

C. Paging only

D. Indexed Addressing requires three accesses

Ans: A

Direct addressing's EA is the address field itself (no extra access), and relative addressing's EA = PC + displacement (also a register-and-field combination, no extra memory access) — both need just one memory reference for the operand itself, unlike indirect addressing's two.

Q245 | Comparison-based · Moderate | Comparative & Mixed Addressing Techniques

Which of the following correctly ranks addressing/translation overhead from LOWEST to HIGHEST memory references typically required (simplest cases)?

A. Indirect < Direct < Indexed

B. Direct/Relative/Indexed (one reference) < Indirect (two references, due to extra pointer fetch)

C. Paging always requires zero references

D. Indirect always requires the fewest references

Ans: B

Direct, relative, and indexed addressing each typically need just one memory access to get the operand, while indirect addressing needs an extra access to first read the pointer — making it comparatively the highest-overhead among these simplest cases.

Q246 | Scenario-based · Moderate | Comparative & Mixed Addressing Techniques

Scenario-based: A compiler needs to generate compact, relocatable branch instructions. Which addressing technique is most appropriate?

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Paging

Ans: C

Compact, relocatable, PC-relative encoding is exactly what relative addressing provides, making it the standard choice for branch instructions that need to work correctly regardless of where the code is loaded.

Q247 | Scenario-based · Easy | Comparative & Mixed Addressing Techniques

Scenario-based: A program processes a large array in a loop, accessing successive elements. Which addressing technique is most appropriate?

A. Direct Addressing

B. Indexed Addressing

C. Indirect Addressing only

D. Paging exclusively

Ans: B

Indexed addressing, with its index register that can be incremented per loop iteration, is the standard mode for stepping through successive array elements.

Q248 | Scenario-based · Moderate | Comparative & Mixed Addressing Techniques

Scenario-based: A program uses a pointer variable whose value (an address) may change at run-time to reference different data. Which addressing technique models this best?

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Indexed Addressing

Ans: B

Indirect addressing — where the address field gives the address of a pointer whose value can change at run-time — models a variable/changeable pointer reference most directly.

Q249 | Conceptual · Moderate | Comparative & Mixed Addressing Techniques

Which of the following addressing techniques is primarily concerned with managing physical vs logical memory mapping at the operating-system/hardware level, rather than operand addressing within a single instruction?

A. Direct Addressing

B. Indexed Addressing

C. Paging

D. Relative Addressing

Ans: C

Paging operates at the level of mapping a process's logical address space onto physical memory frames (a system/hardware-level concern), distinct from Direct, Indexed, and Relative addressing, which determine an operand's address within a single instruction.

Q250 | True-False MCQ · Easy | Comparative & Mixed Addressing Techniques

True/False as MCQ: 'All five addressing/translation techniques discussed (Direct, Indirect, Relative, Paging, Indexed) involve computing some form of effective or physical address from given information.' This statement is:

A. True

B. False

C. True only for Direct and Indirect

D. True only for Paging

Ans: A

All five techniques ultimately compute some address (effective or physical) from given information — this is their shared underlying purpose, even though their mechanisms differ.

Q251 | Comparison-based · Moderate | Comparative & Mixed Addressing Techniques

Which two addressing techniques among those listed both involve an 'addition' operation to compute the effective address?

A. Direct and Indirect

B. Relative and Indexed (PC+offset, Base+Index respectively)

C. Indirect and Paging only

D. None of the techniques use addition

Ans: B

Relative addressing adds a displacement to the PC, and indexed addressing adds an index register to a base — both rely on an addition operation to form the effective address, unlike direct (no computation) or indirect (pointer dereference).

Q252 | **Assertion-Reason · Difficult** | **Comparative & Mixed Addressing Techniques**

Assertion (A): Indirect Addressing and Paging both can involve more than one memory/table access to resolve a final address.
Reason (R): Indirect addressing fetches a pointer before the operand; paging looks up a page table before accessing the physical frame.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Indirect addressing requires reading the pointer location before the operand (an extra access), and paging requires looking up the page table before reaching the physical frame (a translation step) — both can involve more than one access/lookup, so R explains A correctly.

Q253 | **Comparison-based · Moderate** | **Comparative & Mixed Addressing Techniques**

Which addressing technique would be LEAST suitable for accessing a fixed, well-known global variable whose address never changes?

A. Direct Addressing

B. Indirect Addressing (adds unnecessary overhead)

C. Either is equally suitable with no trade-off

D. Relative Addressing is mandatory

Ans: B

For a fixed, unchanging global variable, going through an extra pointer indirection (indirect addressing) adds unnecessary overhead compared to simply using direct addressing, which reaches the variable in one step.

Q254 | **Comparison-based · Difficult** | **Comparative & Mixed Addressing Techniques**

Comparison-based: Which pair of addressing techniques both rely on a register (other than PC) whose value is combined with an instruction-supplied value to form the effective address?

A. Direct and Relative

B. Indexed Addressing uses an index register; Indirect Addressing typically does not need an extra register, relying instead on a memory-held pointer

C. Paging and Direct Addressing

D. None of the techniques use any register

Ans: B

Indexed addressing explicitly uses an index register added to a base value, while indirect addressing relies on dereferencing a memory-held pointer rather than an extra register — this is the structural distinction between the two.

Q255	Comparison-based · Moderate	Comparative & Mixed Addressing Techniques

Which statement best summarizes the trade-off between Direct and Indirect Addressing?

A. Direct addressing trades a smaller address range for faster access; Indirect addressing trades slower access for a larger effective address range

B. Both provide identical address ranges and speed

C. Direct addressing is always slower

D. Indirect addressing never accesses memory

Ans: A

Direct addressing accesses the operand in one step but is limited by the address field's width; indirect addressing needs an extra access but can reach a much larger effective address space via the pointed-to word — this captures their core trade-off.

Q256	Comparison-based · Moderate	Comparative & Mixed Addressing Techniques

Which statement best summarizes the trade-off between Relative and Indexed Addressing?

A. Relative addressing computes EA relative to the current instruction (PC), useful for branching; Indexed addressing computes EA relative to a base using an index register, useful for array access

B. Both are identical in purpose and computation

C. Relative addressing is used only for arrays

D. Indexed addressing is used only for branching

Ans: A

Relative addressing measures EA from the current instruction (PC), ideal for relocatable branches, while indexed addressing measures EA from a data structure's base using an index register, ideal for array element access — these are their respective specialties.

Q257	Conceptual · Moderate	Comparative & Mixed Addressing Techniques

In terms of the instruction execution cycle, addressing modes such as Direct, Indirect, Relative and Indexed are primarily resolved during which phase(s)?

A. Fetch phase only

B. Decode and/or Execute phase, when the effective address is computed

C. Interrupt phase only

D. Reset phase only

Ans: B

The specific addressing mode (direct, indirect, relative, indexed) is identified and acted upon when the instruction is decoded, and the actual effective-address arithmetic is then carried out during decode/execute, not during fetch.

Q258 | Conceptual · Moderate | Comparative & Mixed Addressing Techniques

Which of the following addressing techniques would generally NOT be classified as a classical 'instruction addressing mode' but rather a memory management scheme implemented largely in hardware/OS?

A. Direct Addressing

B. Indexed Addressing

C. Paging

D. Relative Addressing

Ans: C

Paging is fundamentally an OS/hardware memory-management and address-translation mechanism, not a per-instruction addressing mode for specifying an operand within a single instruction, unlike Direct, Indexed, and Relative addressing.

Q259 | Application-based · Moderate | Comparative & Mixed Addressing Techniques

Application-based: A 16-bit address field is used. Using Direct Addressing, what is the maximum number of distinct memory locations directly addressable?

A. 16

B. 256

C. 65536

D. Unlimited

Ans: C

A 16-bit address field can represent 2^16 = 65,536 distinct addresses, so that is the maximum number of directly addressable memory locations.

Q260 | Application-based · Difficult | Comparative & Mixed Addressing Techniques

Application-based: Using Indirect Addressing with a 16-bit address field that points to a memory word capable of storing a 32-bit address, what is the maximum addressable memory range (using the pointed-to word)?

A. 65,536 locations (limited by the 16-bit field)

B. Up to 2^32 locations (limited by the pointed-to word's width, not the 16-bit field)

C. Exactly 32 locations

D. Cannot exceed the direct addressing range

Ans: B

Because the pointed-to memory word (not the 16-bit address field) actually stores the final address, its width (e.g., 32 bits) determines the addressable range, allowing up to 2^32 locations to be reached via indirection.

Q261 | Conceptual · Moderate | Comparative & Mixed Addressing Techniques

Which of the following is TRUE regarding the relationship between Paging and fragmentation?

A. Paging causes external fragmentation but not internal fragmentation

B. Paging avoids external fragmentation but may cause internal fragmentation in the last frame

C. Paging eliminates all forms of fragmentation

D. Paging is unrelated to fragmentation concepts

Ans: B

Fixed-size frames mean any free frame fits any page, eliminating external fragmentation, but a process's last page is often only partially used, causing internal fragmentation — both effects can coexist under paging.

Q262 | Comparison-based · Moderate | Comparative & Mixed Addressing Techniques

Which addressing technique's effective address calculation is LEAST dependent on any register value being added or used as a pointer, relying purely on the instruction's address field?

A. Direct Addressing

B. Indirect Addressing

C. Relative Addressing

D. Indexed Addressing

Ans: A

Direct addressing's effective address is taken purely from the instruction's address field with no register or pointer involved, making it the least dependent on any register among the listed techniques.

Q263 | Scenario-based · Moderate | Comparative & Mixed Addressing Techniques

Scenario-based: A processor architecture wants to support both efficient array access and relocatable branch code in the same instruction set. Which two addressing techniques would it most likely include for these respective purposes?

A. Direct Addressing for arrays; Paging for branches

B. Indexed Addressing for arrays; Relative Addressing for branches

C. Indirect Addressing for arrays; Direct Addressing for branches

D. Paging for arrays; Indexed Addressing for branches

Ans: B

Indexed addressing efficiently supports array/loop access via an incrementable index register, while relative addressing efficiently supports relocatable branch instructions via PC-relative offsets — pairing each technique with its natural strength.

Q264 | Assertion-Reason · Moderate | Comparative & Mixed Addressing Techniques

Assertion (A): The instruction execution cycle's decode phase determines which addressing mode (direct, indirect, relative, indexed) applies to a given instruction.
Reason (R): The opcode/instruction format typically encodes the addressing mode to be used for effective address computation.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

The decode phase identifies the addressing mode encoded in the instruction's opcode/format, and based on that, the appropriate effective-address computation (R) is triggered — A is true and R correctly explains it.

Q265 | Match-the-Following · Easy | Comparative & Mixed Addressing Techniques

Which of the following correctly pairs each addressing technique with its PRIMARY typical use case? (1) Direct - fixed variables, (2) Indirect - pointers, (3) Relative - branching, (4) Indexed - arrays, (5) Paging - virtual memory management

A. All pairings are correct

B. Only pairings (1) and (2) are correct

C. Only pairings (3) and (4) are correct

D. None of the pairings are correct

Ans: A

Each technique's typical primary use matches its computational mechanism: Direct→fixed variables, Indirect→pointers, Relative→branching, Indexed→arrays, Paging→virtual memory management — all five pairings are textbook-correct.

Q266 | Statement-based · Moderate | Comparative & Mixed Addressing Techniques

Which of the following statements is FALSE?

A. Direct addressing requires only one memory reference for the operand

B. Indirect addressing always requires fewer memory references than direct addressing

C. Relative addressing uses the PC to compute effective address

D. Indexed addressing uses an index register to compute effective address

Ans: B

Indirect addressing actually requires MORE memory references than direct addressing (two vs one), not fewer, so the claim that it 'always requires fewer' is false — making this the false statement.

Q267 | Comparison-based · Moderate | Comparative & Mixed Addressing Techniques

Which of the following correctly ranks Direct, Indirect, Relative, and Indexed addressing by whether they require an explicit register (other than PC) to compute the effective address?

A. Only Indexed addressing requires an extra register (the index register); Direct, Indirect and Relative do not

B. All four require an extra register

C. None of the four require any register

D. Only Direct addressing requires an extra register

Ans: A

Among the four instruction-level modes, only Indexed Addressing explicitly requires a dedicated extra register (the index register) added to a base value; Direct uses only the address field, Indirect uses a memory-held pointer, and Relative uses the PC.

Q268 | Scenario-based · Moderate | Comparative & Mixed Addressing Techniques

Scenario-based: A processor's instruction set includes Direct, Indirect, Relative and Indexed addressing modes. A programmer needs to implement a subroutine that processes a table of records and also needs to branch back to the loop's start each iteration. Which TWO addressing modes are most relevant respectively for these two needs?

A. Direct Addressing for the table; Indirect Addressing for the branch

B. Indexed Addressing for the table; Relative Addressing for the branch

C. Paging for the table; Indexed Addressing for the branch

D. Indirect Addressing for the table; Direct Addressing for the branch

Ans: B

A table of records is naturally traversed using an incrementable index register (Indexed Addressing), while looping back to the start of code is naturally expressed as a small PC-relative offset (Relative Addressing).

Q269 | Assertion-Reason · Difficult | Comparative & Mixed Addressing Techniques

Assertion (A): Paging is unique among the five techniques discussed in that it operates transparently to most user-level instructions, rather than being explicitly selected per instruction.
Reason (R): Paging is implemented by the memory management unit (MMU)/OS as a background address-translation layer, not as an addressing mode encoded in each instruction's opcode.
Choose the correct option:

A. Both A and R true, R explains A

B. Both A and R true, R does not explain A

C. A true, R false

D. A false, R true

Ans: A

Paging works behind the scenes via the MMU/OS to translate addresses, rather than being a mode a programmer selects per instruction the way Direct, Indirect, Relative, or Indexed addressing are — R correctly explains why paging is structurally different.

Q270 | **Comparison-based · Moderate** | **Comparative & Mixed Addressing Techniques**

Comparison-based: Among Direct, Indirect, Relative, and Indexed addressing, which pair shares the property that effective address computation involves simple addition of two known register/field values (no memory dereference)?

A. Direct and Indirect

B. Relative and Indexed

C. Direct and Relative

D. Indirect and Indexed

Ans: B

Relative addressing computes EA = PC + displacement, and Indexed addressing computes EA = Base + Index register — both involve a straightforward addition of two known values with no extra memory dereference, unlike Indirect (which dereferences a pointer) or Paging (which performs a table lookup).

Q271 | **Statement-based · Difficult** | **Comparative & Mixed Addressing Techniques**

Statement-based: Consider the following statements about all five techniques together:
(i) Only indirect addressing and paging may require more than one memory/table access to fully resolve an address.
(ii) Direct, relative, and indexed addressing each compute EA using only register/field values plus at most one final memory access for the operand itself.
(iii) All five techniques are mutually exclusive and can never be combined in the same architecture.
Which statements are correct?

A. (i) and (ii) only

B. (i), (ii) and (iii)

C. (ii) and (iii) only

D. (i) and (iii) only

Ans: A

Indirect addressing and paging both can require more than one memory/table access to fully resolve an address (i true), while Direct, Relative and Indexed each need at most one memory access for the operand itself after computing EA via registers/fields (ii true); however, real architectures CAN and do combine modes (e.g., indexed-indirect), so (iii) is false.

Q272 | **Application-based · Difficult** | **Comparative & Mixed Addressing Techniques**

Which addressing technique would be most natural for a compiler implementing access to a local variable stored at a fixed offset from the current stack frame, if a dedicated 'base/frame' register is used instead of PC?

A. This is essentially an indexed-addressing-like calculation, using the frame/base register similarly to an index register

B. This must always be relative addressing using the PC

C. This must always be paging

D. This is impossible without indirect addressing

Ans: A

Using a dedicated base/frame register added to a fixed offset to reach a local variable is structurally the same addition-based computation as indexed addressing, just using a frame register instead of a general index register.

Q273 | True-False MCQ · Easy | Comparative & Mixed Addressing Techniques

True/False as MCQ: 'Every addressing technique among Direct, Indirect, Relative, Indexed, and Paging ultimately serves the same underlying purpose: determining where in memory to find or store data.' This statement is:

A. True

B. False

C. True only for Direct and Indirect

D. True only for Paging

Ans: A

Despite their differing mechanisms, Direct, Indirect, Relative, Indexed addressing, and Paging all ultimately exist to answer the same basic question: where in memory should this operand be read from or written to.

Q274 | Conceptual · Difficult | Comparative & Mixed Addressing Techniques

Which of the following is the most accurate way to describe the relationship between addressing modes (Direct/Indirect/Relative/Indexed) and paging in a real system?

A. They are mutually exclusive; a system can use only one or the other

B. They typically coexist: an instruction's addressing mode computes a logical/virtual effective address, which paging then translates into a physical address

C. Paging replaces the need for any addressing mode

D. Addressing modes only apply after paging has translated the address

Ans: B

In practice, an instruction's addressing mode first computes a logical (virtual) effective address; if the system uses paging, that logical address is then separately translated by the MMU into a physical address — the two mechanisms operate at different layers and typically work together, not as alternatives.

Q275 | Comparison-based · Difficult | Comparative & Mixed Addressing Techniques

Comparison-based: Which two addressing techniques are most directly suited to implementing relocatable code, and why?

A. Direct and Indirect, because they use absolute addresses

B. Relative Addressing (PC-relative offsets) and Paging (logical-to-physical translation independent of absolute load address), because both decouple the program's internal references from one fixed physical location

C. Indexed and Direct, because they use index registers

D. None of the techniques support relocation

Ans: B

Relative addressing keeps branch targets valid regardless of where code is loaded (since they're PC-relative), and paging keeps a process's logical addresses valid regardless of where its pages are physically placed (since the page table handles translation) — both decouple logical references from a fixed physical location, supporting relocation.

Answer Key — Quick Reference

Q1: **B**	Q2: **B**	Q3: **C**	Q4: **B**	Q5: **C**	Q6: **C**	Q7: **B**	Q8: **B**
Q9: **B**	Q10: **B**	Q11: **D**	Q12: **D**	Q13: **A**	Q14: **A**	Q15: **B**	Q16: **A**
Q17: **C**	Q18: **D**	Q19: **B**	Q20: **B**	Q21: **B**	Q22: **C**	Q23: **D**	Q24: **B**
Q25: **A**	Q26: **A**	Q27: **A**	Q28: **B**	Q29: **B**	Q30: **B**	Q31: **A**	Q32: **B**
Q33: **B**	Q34: **C**	Q35: **A**	Q36: **B**	Q37: **B**	Q38: **A**	Q39: **B**	Q40: **B**
Q41: **A**	Q42: **A**	Q43: **B**	Q44: **C**	Q45: **A**	Q46: **B**	Q47: **A**	Q48: **B**
Q49: **A**	Q50: **B**	Q51: **B**	Q52: **B**	Q53: **B**	Q54: **B**	Q55: **A**	Q56: **B**
Q57: **A**	Q58: **A**	Q59: **A**	Q60: **B**	Q61: **B**	Q62: **B**	Q63: **B**	Q64: **B**
Q65: **B**	Q66: **B**	Q67: **A**	Q68: **B**	Q69: **B**	Q70: **B**	Q71: **B**	Q72: **B**
Q73: **C**	Q74: **A**	Q75: **B**	Q76: **A**	Q77: **B**	Q78: **B**	Q79: **A**	Q80: **C**
Q81: **B**	Q82: **A**	Q83: **B**	Q84: **B**	Q85: **A**	Q86: **B**	Q87: **C**	Q88: **C**
Q89: **C**	Q90: **B**	Q91: **B**	Q92: **B**	Q93: **B**	Q94: **B**	Q95: **A**	Q96: **A**
Q97: **B**	Q98: **B**	Q99: **B**	Q100: **A**	Q101: **A**	Q102: **A**	Q103: **B**	Q104: **B**
Q105: **B**	Q106: **B**	Q107: **A**	Q108: **B**	Q109: **B**	Q110: **B**	Q111: **C**	Q112: **A**
Q113: **B**	Q114: **B**	Q115: **A**	Q116: **B**	Q117: **C**	Q118: **A**	Q119: **A**	Q120: **B**
Q121: **B**	Q122: **A**	Q123: **A**	Q124: **B**	Q125: **A**	Q126: **B**	Q127: **B**	Q128: **B**
Q129: **B**	Q130: **B**	Q131: **D**	Q132: **B**	Q133: **B**	Q134: **A**	Q135: **B**	Q136: **B**
Q137: **B**	Q138: **B**	Q139: **A**	Q140: **B**	Q141: **A**	Q142: **A**	Q143: **C**	Q144: **B**
Q145: **B**	Q146: **B**	Q147: **B**	Q148: **B**	Q149: **B**	Q150: **B**	Q151: **A**	Q152: **D**
Q153: **B**	Q154: **A**	Q155: **A**	Q156: **A**	Q157: **B**	Q158: **B**	Q159: **B**	Q160: **B**
Q161: **A**	Q162: **B**	Q163: **B**	Q164: **B**	Q165: **B**	Q166: **B**	Q167: **B**	Q168: **A**
Q169: **B**	Q170: **B**	Q171: **B**	Q172: **A**	Q173: **A**	Q174: **A**	Q175: **A**	Q176: **B**
Q177: **B**	Q178: **B**	Q179: **B**	Q180: **B**	Q181: **A**	Q182: **A**	Q183: **A**	Q184: **A**
Q185: **A**	Q186: **A**	Q187: **A**	Q188: **A**	Q189: **B**	Q190: **A**	Q191: **C**	Q192: **C**
Q193: **B**	Q194: **A**	Q195: **A**	Q196: **B**	Q197: **A**	Q198: **B**	Q199: **A**	Q200: **A**
Q201: **A**	Q202: **B**	Q203: **B**	Q204: **C**	Q205: **B**	Q206: **B**	Q207: **A**	Q208: **A**
Q209: **B**	Q210: **B**	Q211: **B**	Q212: **A**	Q213: **B**	Q214: **A**	Q215: **B**	Q216: **A**
Q217: **B**	Q218: **A**	Q219: **D**	Q220: **C**	Q221: **B**	Q222: **A**	Q223: **C**	Q224: **A**
Q225: **A**	Q226: **B**	Q227: **B**	Q228: **C**	Q229: **B**	Q230: **A**	Q231: **A**	Q232: **A**
Q233: **A**	Q234: **B**	Q235: **B**	Q236: **A**	Q237: **A**	Q238: **C**	Q239: **B**	Q240: **C**
Q241: **B**	Q242: **A**	Q243: **A**	Q244: **A**	Q245: **B**	Q246: **C**	Q247: **B**	Q248: **B**
Q249: **C**	Q250: **A**	Q251: **B**	Q252: **A**	Q253: **B**	Q254: **B**	Q255: **A**	Q256: **A**
Q257: **B**	Q258: **C**	Q259: **C**	Q260: **B**	Q261: **B**	Q262: **A**	Q263: **B**	Q264: **A**
Q265: **A**	Q266: **B**	Q267: **A**	Q268: **B**	Q269: **A**	Q270: **B**	Q271: **A**	Q272: **A**
Q273: **A**	Q274: **B**	Q275: **B**					